

Forging 1024-bit RSA signatures in nearly SNFS time

Laura Shea¹, Miro Haller¹, Adam Suhl¹,
Nadia Heninger¹, and Emmanuel Thomé²

¹ University of California San Diego

² Inria Nancy

{lmshea,mhaller,asuhl,nadiiah}@ucsd.edu, emmanuel.thome@inria.fr

September 20, 2026

Abstract. The security of RSA is generally understood to be based on the complexity of factoring, and key size parameters are extrapolated from the general number field sieve (GNFS). However, this may not accurately represent RSA security in practical scenarios.

An under-appreciated 2007 algorithm of Joux, Naccache, and Thomé allows an attacker to forge RSA signatures after temporary access to a raw RSA signing/decryption oracle in time close to the *special* number field sieve (SNFS) without factoring the key. We implement and run this algorithm for 1024-bit RSA. In total, the attack took 1380 CPU core-years over five calendar months, and made 2^{32} oracle queries. Most of this time is precomputation; after the precomputation the attacker can forge any signature of choice, offline, in 180 core-years.

We carried out our attack using a hardware security module (HSM) as the signing oracle, thus demonstrating the ability to impersonate the HSM through black-box API interactions, without exfiltrating the key. Blind RSA schemes also provide such a signing oracle.

Extrapolating our empirical running times to larger key sizes, we conclude that the concrete security of RSA with a signing oracle should be 15 to 30 bits lower than the factoring-based security estimates for the 1024-bit to 4096-bit RSA parameters that are common in practice. Even 4096-bit RSA does not appear to meet a 128-bit security level in this attack model. This highlights a gap in current RSA-type security assumptions, and gives classical cryptanalytic evidence in favor of moving away from RSA entirely during the current post-quantum transition.

1 Introduction

In this paper, we carry out an attack that allows us to forge arbitrary 1024-bit RSA digital signatures (or decrypt arbitrary ciphertexts) using academic computational resources, without factoring the modulus. This attack reduces RSA

Alternative title: Nearly Special-Number-Field-Sieve-Speed Signature Forgery Sans Factoring N (NSNFSSSFSFN)

security levels below acceptable thresholds for modern cryptographic deployments, and demonstrates a theoretical gap in RSA-based security assumptions and practical parameter choices.

The threat model we consider is a “lunchtime attack,” or a non-adaptive chosen ciphertext attack (IND-CCA1)—an attacker who obtains temporary access to an RSA signature/decryption oracle for a given key, and then is challenged to generate a signature forgery or ciphertext decryption after losing access. We demonstrate that temporary oracle access, together with much more modest computational resources than factoring the modulus would require, can give the attacker the permanent ability to forge arbitrary signatures for that key pair offline. That is, the attacker can effectively use oracle queries to steal abilities equivalent to possessing the private key, without actually computing it.

The attack runs in two separate stages: an initial precomputation and oracle query phase, followed by a fully offline signature forgery/decryption phase that can be repeated at will for arbitrary targets and requires no further oracle queries. The precomputation stage depends only on the modulus N and took us about 1,200 core-years to complete. The oracle query phase required us to query around 2^{32} raw, unpadded RSA signatures/decryptions. We carried out our oracle queries using an HSM (Hardware Security Module). The final offline phase takes around 180 core-years to forge any signature (or target decryption).

In comparison, current estimates for factoring a 1024-bit RSA modulus would take 500,000–1,000,000 CPU core-years with current implementations [15]. The computation time for our attack can almost certainly be improved, and should be interpreted as an upper bound.

The attack we implement is from a 2007 paper of Joux, Naccache, and Thomé [42]. Their attack runs in asymptotic time $L_N(1/3, 1.577 + o(1))$, close to that of the special number field sieve. This is significantly less than the $L_N(1/3, 1.923 + o(1))$ running time of the general number field sieve for factoring. We implemented several algorithmic improvements and had to overcome a number of computational challenges to carry out this attack for 1024-bit RSA.

We detail the implications for both practical and theoretical settings below.

1.1 Practical impacts

RSA digital signatures are still in wide use for important protocols like TLS and OAuth [13], with 2048-bit keys the most common choice. RSA’s days are numbered, however: NIST plans to deprecate it by 2030 and disallow it by 2035 in favor of post-quantum algorithms [54], and the transition to hybrid and post-quantum algorithms is underway. Although NIST disallowed 1024-bit RSA in 2013 for signature generation [6] and modern implementations have largely migrated to 2048 bits, 1024-bit keys are still allowed for signature validation under legacy use [7], and remain in use today. (See Section 7.4.)

While the hardness of factoring is generally used to determine RSA key strengths (e.g. in NIST SP 800-56B Rev. 2 [53]), there are several practical usage scenarios that match the signing oracle access attack model we consider in this paper. Such an attack likely does not pose an immediate operational threat

to most modern RSA deployments, but it demonstrates that RSA provides significantly less security than the factoring-based security estimates would suggest in these settings.

Security estimates. 1024-bit RSA is generally considered to provide 80 bits of security; we estimate that the algorithm we implement has 2^{65} running time for 1024-bit RSA. Scaling the asymptotic running time to 2048-bit RSA gives a 2^{90} running time for the attack, with 2^{43} queries, a significant reduction from the 112-bit security that 2048-bit RSA is generally understood to provide.³ Even 4096-bit RSA fails to achieve 128 bits of security in this attack model—we extrapolate that such an attack would take 2^{119} computation time and require 2^{57} queries. See Section 6 for more details.

Hardware security modules. Hardware security modules (HSMs) are often used to protect secret keys and accelerate cryptographic operations in enterprise environments. Current security auditing best practices, as well as standards such as PCI DSS [57], encourage or require the use of HSMs to protect key material. Standards such as FIPS 140-3, ISO/IEC 19790, ISO/IEC 24759, and ISO/IEC 15408 (Common Criteria) define requirements for these types of cryptographic modules to be tamper-proof and include side-channel countermeasures to protect keys even against an attacker with physical access.

The PKCS 11 standard [55] implemented by nearly all HSMs provides an interface for raw RSA operations that exactly matches the oracle query functionality required by our attack. Thus, in a threat model requiring an HSM to protect private keys in such a scenario, our attack can be viewed as allowing an attacker to effectively extract capabilities equivalent to the private key in a feasible amount of time with only *black-box access* to the HSM itself.

Blind signature schemes. RSA-based blind signature schemes also provide the type of signature query oracle required by our attack, and thus this attack should be interpreted as lowering the bit security of RSA in protocols using blind signatures. We analyze some deployed applications in Section 7.2 and observe that the 2048-bit key sizes common in practice provide too low of a security threshold.

1.2 Theoretical impacts

The problem of breaking RSA (that is, computing e th roots modulo an integer N of unknown factorization) has not been shown to be equivalent to factoring N in the nearly 50 years since the invention of the RSA cryptosystem. Results in various models [1] have not been shown to apply to $\mathbb{Z}_N^*$.

There are a number of computational assumptions associated with RSA security. A prominent one for signatures is the one-more RSA assumption [8].

³ For purposes of comparison, the Bitcoin blockchain’s hash rate as of this writing in September 2026 is 900Ehash/s, which translates to around 2^{90} hashes every two weeks. Cloudflare has stated that they handle more than 2^{43} HTTP requests per day [10].

However, we observe that the attack we implement, which we name $\sqrt[n]{\text{NFS}}$, does not align with one-more RSA or variants despite being a clear practical threat.

We propose that there is a gap both in the practice of using the running time of GNFS to set parameter sizes for these RSA-type assumptions, and in the assumptions themselves. We do not resolve this gap. We suggest that pre-computation attacks should be captured in RSA-type assumptions, and that subexponential-time attacks (including their lower-order terms) should be reasonably analyzable in those frameworks. We have named our attack setting “delayed-target RSA” in line with the naming scheme of Koblitz and Menezes [45]. We discuss this in detail in Section 5.

1.3 RSA progress in general

The algorithmic result we use has been known for two decades but the impact on RSA parameter choices does not appear to have been understood in the community. We hope that our concrete implementation for a landmark key size makes the difference much clearer. 1024-bit RSA has long been considered a minimal security level, and remains in use today in some specialized cases.

Coincidentally, the factorization of RSA-260, along with that of a 1277-bit special modulus, was publicly posted [48] a few days after our own computation finished. At least two individuals had been independently working on RSA-260, using AI to translate **CADO-NFS** to run on GPUs, and then using GPU compute from their AI company employers to run it [70]. Both our findings and the pre-2026 factoring records were completed using academic computational resources and engineering skills, a far cry from industry- or state-level adversaries.

That leaves substantial leeway before one can get a sense of the feasibility limit that would apply to adversaries with larger computing power. Extrapolations that RSA-1024 could be factored in weeks using industry-grade GPU facilities are no fantasy. The important takeaway is that improvements to sieving for the GNFS, including to ECM or sieving on GPUs, can definitely be obtained. Any speed improvements to **CADO-NFS** would apply in a straightforward way to the $\sqrt[n]{\text{NFS}}$ algorithm as well. Weis and Lu report that factoring RSA-260 cost 25 to 13.5 GPU-years (respectively) using their implementations. A back-of-the-envelope scaling suggests that our 1024-bit RSA signature forgery computation would take between 2 and 9 GPU-years if we had used their implementations; for 2048 bits a very rough estimate would be 2^{27} GPU-years.

NFS development. **CADO-NFS** [64] is an open-source implementation of the number field sieve algorithms for factoring and discrete logarithm that has been used for several recent computational records. The algorithm that we implement did not have a public implementation prior to our work. Our implementation makes use of the highly optimized **CADO-NFS** code for sieving, linear algebra, and descent, among other algorithmic procedures, but uses them in a different setting, in a different order and with different parameters.

Our engineering work does not hugely transform the **CADO-NFS** codebase we started with—at least for the usual number field sieve algorithms. However it

took significant technical effort to port **CADO-NFS** to our setting, and our work could be useful for other such extensions to the NFS. The progression of our 1024-bit computation—from parameter selection to **CADO-NFS** roadblocks to algorithmic changes—will be an informative data point for future large computations. In particular, we implemented interesting variations of algorithms for descent, and for e th roots in a number field.

The future of RSA. The community has been focusing on quantum computing attacks as a reason to retire RSA, and the classical cryptanalysis of RSA has been largely regarded as settled for decades, with occasional factoring records set by teams that track with general-purpose computing improvements over the years. We hope that our computation here illustrates that there are still surprises to be found in the *classical* cryptanalysis of RSA, and if more attention had been paid there would have been even stronger reasons to retire RSA earlier.

The security of RSA and the family of NFS-based algorithms used to calculate RSA key sizes should not be viewed as a static monolith. Even the classical security of RSA is more complex than is generally appreciated in the community.

While we do not believe that the attack we implement should be cause for immediate alarm among practitioners managing cryptographic inventory in industry—it is practical in an academic sense rather than in the script-kiddie sense, and the attack model of a raw, unpadded RSA signing oracle is very strong in practice even if academically relevant—we believe that this attack demonstrates that 2048 to 4096-bit RSA falls short of modern cryptographic security levels requiring 128 bits of security or larger. Given this, the current transition to post-quantum algorithms presents an opportunity to transition to modern algorithms and security levels in the next few years.

1.4 Summary of computational results

We summarize the results of our large computations in Figure 1. Our code is available at <https://github.com/ucsd-hacc/NSNFSSFSFN>.

Key length	Precomputation (core-years)	Queries	Individual forgery (core-years)
512	0.01	2^{23}	0.001
666	3.5	2^{27}	0.07
768	56	2^{27}	42
1024	1200	2^{32}	180

Fig. 1. Empirical running times for $\sqrt[e]{\text{NFS}}$. We experimented with different parameter settings giving different tradeoffs for different phases of the algorithm.

Algorithm	Complexity	Modulus size	Year, citation	CPU time	Wall time
GNFS-DL	G	795	2019 [14]	3200 core-years	
GNFS	G	795	2019 [14]	1000 core-years	
GNFS	G	829	2020 [14]	2700 core-years	
SNFS-DL	S	1024	2016 [28]	385 core-years	2 months
SNFS	S	1039	2007 [3]	225 core-years	> 6 months
GNFS	G (gpu)	862	2026 [48]	13.5 GPU-years	
GNFS	G (gpu)	896	2026 [71]	TBA	
SNFS	S (gpu)	1277	2026 [48]	6.6 GPU-years	
$\sqrt[5]{\text{NFS}}$	S+	512	2007 [42]	> 2 core-days	
$\sqrt[5]{\text{NFS}}$	S+	512	(this work)	4.8 core-days	70 minutes
$\sqrt[5]{\text{NFS}}$	S+	666	(this work)	3.6 core-years	33 hours
$\sqrt[5]{\text{NFS}}$	S+	768	(this work)	98 core-years	3 weeks
$\sqrt[5]{\text{NFS}}$	S+	1024	(this work)	1380 core-years	5 months

Fig. 2. Related record computations for factoring, discrete logarithm, and $\sqrt[5]{\text{NFS}}$. We categorize complexity as either General (G), Special (S), or slightly above special (S+).

2 Preliminaries

Number field sieve. The fastest known classical general-purpose algorithms for factoring and discrete log are a family of Number Field Sieve (NFS) algorithms. To disambiguate, we will refer specifically to the GNFS and SNFS variants, and their discrete log variants GNFS-DL and SNFS-DL. The “G” variants work for a *general* modulus, meaning any generic integer modulus. The “S” variants only work for a *special* modulus—in brief, a modulus that admits an efficiently discoverable number field where sieving is faster.

The topic of this paper is what we call the $\sqrt[5]{\text{NFS}}$ algorithm (“eth root NFS”). It was introduced in 2007 by Joux, Naccache, and Thomé (JNT) [42, §4]. $\sqrt[5]{\text{NFS}}$ is not a straightforward extension of the usual GNFS or GNFS-DL algorithms. While it reuses large parts of them, the steps are slightly different, and the parameter tradeoffs differ even further. We will elaborate on this later; for now we stress that $\sqrt[5]{\text{NFS}}$ should be viewed as a different algorithm in the NFS family. For a readable but exhaustive resource on the NFS, we refer the reader to the lecture notes and references in [69].

Figure 2 lists some record computations along with our own results. An up-to-date list is often kept on Wikipedia [72]. We have omitted the breakdown of precomputation/individual computation in this table; there is a similar division for the discrete log variants but not for factoring.

Subexponential complexity. The running times of all of the algorithms we discuss in this paper are subexponential, with running time expressed asymptotically as

$L_N(\alpha, c)^{1+o(1)}$ with $L_N(\alpha, c) = \exp(c \cdot (\ln N)^\alpha (\ln \ln N)^{1-\alpha})$. In this paper, we will always encounter $\alpha = 1/3$ with a varying c .

The running time of the GNFS for factoring is $L_N(1/3, (64/9)^{1/3} + o(1))$ with N the integer to be factored, and $c \approx 1.923$. This is the fastest known asymptotic running time to factor generic integers, such as well-generated RSA moduli. The running time of the GNFS-DL for prime field discrete logarithm is the same, with N the prime modulus.

The running time of the SNFS for factoring is $L_N(1/3, (32/9)^{1/3} + o(1))$, which improves $c \approx 1.526$. The SNFS can be used to factor integers that are efficiently expressible in a special form; for example it applies to integers close to a power of 2. To give an intuitive sense of this difference, a 1024-bit RSA modulus has yet to be factored publicly, but a 1039-bit Mersenne number was publicly factored using the SNFS algorithm in 2007 with 225 core-years of compute effort [3]. A similar gap exists in the discrete logarithm case [28].

For our purposes, “nearly special” complexity is $L_N(1/3, 1.577 + o(1))$. This is the overall complexity of the $\sqrt[e]{N}$ FS algorithm when a practical variant called extension sieving is used. (Which we did.) The full statement of the running time, along with the proof, were given in [42].

Our problem setting. The most apt name we have seen for our RSA problem of interest is “delayed-target RSA” following the terminology of Koblitiz and Menezes [45] when discussing JNT’s work. We give some more detail and context for this in Section 5. For now, we describe the problem as follows. An attacker is assumed to operate in two phases, sharing state between them. In the first phase, they are given an RSA public key (N, e) and may query an oracle which, on input w , returns $w^{1/e} \bmod N$. (In usual notation, $1/e = d \bmod \varphi(N)$.) In the second phase, the attacker receives a target $t \leftarrow \mathbb{Z}_N^*$ and loses access to the oracle. They must produce $t^{1/e} \bmod N$ to win the game.

It turns out that, as currently known, an attack on delayed-target RSA does not yield an attack for usual RSA (without the oracle), nor for factoring. There is a more subtle relationship to one-more RSA [8], a more popular assumption used to prove security of blind signatures. We discuss that relationship in Section 5.

Other similar attack settings. Problem definitions like delayed-target RSA have made some appearances in the literature. On the attack side, JNT considered our delayed-target RSA as well as variants for other padding schemes [42]. Joux, Lercier, Naccache and Thomé [41] considered delayed-target CDH in finite-field groups, where a similar attack applies. Koblitiz and Menezes [45] gave attacks for delayed-target DL on hyperelliptic curves. On the constructive side, delayed-target CDH was used to prove security of Freeman’s identification scheme [27]. Delayed-target DDH has appeared in proofs for the CCA1 security of ElGamal [47] and Cramer-Shoup [9]. We are not aware of any proofs relying on delayed-target RSA; if any exist they are likely under a different name.

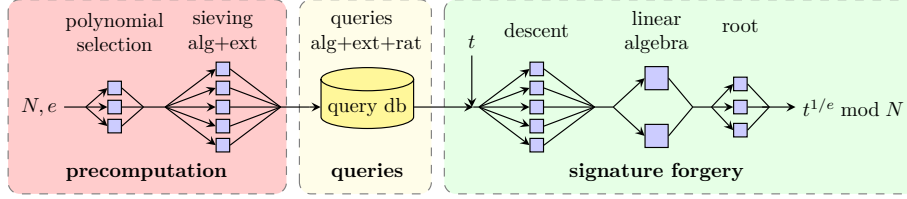


Fig. 3. The $\sqrt[e]{\text{NFS}}$ algorithm runs in three main stages: a precomputation step that depends only on the public modulus and exponent, an online query phase that depends on the output of the precomputation, and finally an individual signature forgery that can be run offline using only the output of the first two stages. The “sieving” stage is by far the most expensive; this diagram is not to scale.

3 The $\sqrt[e]{\text{NFS}}$ algorithm, with our additions

The $\sqrt[e]{\text{NFS}}$ algorithm is from Section 4 of JNT [42]. That paper refers to the “one-more RSA inversion” problem but to be technically precise, it is attacking delayed-target RSA as we defined in the Preliminaries.

Our implementation of $\sqrt[e]{\text{NFS}}$ uses **CADO-NFS** [64], **SageMath** [67], **PARI** [66] and additional code written in Python. It makes heavy use of existing NFS ideas and folklore. The major modifications and new techniques are discussed in this section, along with enough background to be understandable⁴. The specific parameters and progression of the 1024-bit computation are discussed in Section 4.

3.1 Bird’s-eye view

The $\sqrt[e]{\text{NFS}}$ algorithm can be viewed as a malleability attack, combining ideas from the GNFS and GNFS-DL algorithms. The overall flow is shown in Figure 3. Broadly, the goal is to be able to express any target t as a product of elements whose e th roots we know from precomputation and oracle queries. Of course, if these “elements” are only rational primes, the running time and success probability are wholly impractical. Rather we develop an equivalence, following the same notations as in [42, §4.1]:

$$t^d = \frac{\left(\frac{u}{v}S(m)\right)^d}{R(m)} \cdot T(m)^d \cdot U(m)^d \quad (1)$$

where m is from our setup; u , v , and S are from descent; T is from precomputation; U is from linear algebra; R is from taking a root in the number field; and all the d th powers are known from precomputed oracle queries. Roughly, lowercase letters are integers and uppercase letters are rational functions.

In a nutshell, the algorithm approaches SNFS complexity because it only needs to sieve for smooth relations on one side, compared to two sides for GNFS.

⁴ We recognize that there are limits to the understandability of NFS-style algorithms, and refer the interested reader to [69] for details and background.

The “other side” is traded for oracle queries. However, the descent phase does *not* become easier, which necessitates rebalancing the steps of the computation.

3.2 Detailed view: Precomputation and oracle queries

Polynomial selection. Given a modulus N , we select two polynomials f, g such that $f(m) = g(m) = 0 \pmod{N}$ for a shared root m . For all relevant cases we ran, an optimal choice of degrees is $\deg(g) = 1$ and $\deg(f) \in \{5, 6\}$. Our number field is then $K = \mathbb{Q}(\alpha)$ where $f(\alpha) = 0$. Its ring of integers is $\mathcal{O}_K$. For later steps⁵ it is useful to select f such that the public RSA exponent e is unramified (or even inert) in K , and there are several other small inert primes. Other than these extra checks, the usual NFS polynomial selection can be used.

There are assumed to be three smoothness parameters input to the algorithm: B_{alg} , B_{ext} , and B_{rat} . Let:

$$\mathcal{F}_{\text{alg}} = \{\mathfrak{p} \mid \mathfrak{p} \subset \mathcal{O}_K \text{ is prime and } \|\mathfrak{p}\| < B_{\text{alg}}\}$$

$$\mathcal{F}_{\text{ext}} = \{\mathfrak{p} \mid \mathfrak{p} \subset \mathcal{O}_K \text{ is prime and } B_{\text{alg}} < \|\mathfrak{p}\| < B_{\text{ext}}\}$$

$$\mathcal{F}_{\text{rat}} = \{p \mid p \in \mathbb{Z} \text{ is prime and } p < B_{\text{rat}}\}$$

We call these sets “factor bases.”

One-sided precomputation sieving. We made slight fixes to the `CADO-NFS 1as` program to allow for one-sided sieving. For $\sqrt[e]{NFS}$, in contrast to the GNFS and the SNFS, precomputation sieving is done only on the algebraic side. It is broken down into two steps. In the first, **algebraic sieving**, sieving is performed to find relations that are B_{alg} -smooth. By “relation” we mean a tuple (a, b) and a factorization such that:

$$(a - b\alpha) = \prod \mathfrak{p}_i^{\mu_i}$$

where $(a - b\alpha)$ is the ideal generated by $a - b\alpha \in K$, and the $\mathfrak{p}_i$ are all in $\mathcal{F}_{\text{alg}}$. We find enough of these algebraic relations such that every $\mathfrak{p}_i$ in $\mathcal{F}_{\text{alg}}$ is included in at least one relation.

The second sieving step is **extension sieving**. For every $\mathfrak{q} \in \mathcal{F}_{\text{ext}}$, special- $\mathfrak{q}$ sieving is performed to find one relation where the factorization includes exclusively $\mathfrak{q}$ and primes $\mathfrak{p}_i$ that are in the smaller $\mathcal{F}_{\text{alg}}$ (or at least of norm less than the norm of $\mathfrak{q}$). In practice, this does not need to occur for every $\mathfrak{q} \in \mathcal{F}_{\text{ext}}$, but it should occur for a good portion of them.

Oracle queries. From the above precomputation, we make **algebraic queries** for all of the algebraic relations, and **extension queries** for all of the extension relations. That is, for a relation (a, b) we make a query to learn $(a - b\alpha)^d \in \mathbb{Z}_N^*$. We also make **rational queries** to learn $p_i^d \in \mathbb{Z}_N^*$ for all rational primes p_i from 2 to B_{rat} . If our polynomial g is not monic (which is nearly always the case), we make queries for all the prime factors of its leading coefficient. Often, these are already covered by the rational queries.

⁵ Due to how `CADO-NFS` computes Schirokauer maps, and for the p -adic root.

3.3 Detailed view: Descent

Three-stage descent. Upon receiving a forgery target t , the goal of descent is to produce an initial split $u/v = t \bmod N$, and a rational function S , such that:

$$(S(\alpha)) = \prod_{\mathfrak{p}_i \in \mathcal{F}_{\text{alg}} \cup \mathcal{F}_{\text{ext}}} \mathfrak{p}_i^{\mu_i} \quad \text{and} \quad u/v \cdot S(m) = \prod_{p_j \in \mathcal{F}_{\text{rat}}} p_j^{\nu_j}$$

In other words, the ideal generated by $S(\alpha)$ is smooth over the entire algebraic side, and $u/v \cdot S(m)$ is smooth over the rational side. We will not beat the reader over the head with the detailed mechanisms of descent, and will simply say that this function S —along with the valuations μ and ν —can be found by iteratively running special- q and special- q sieving to find increasingly smooth *two-sided* relations. A two-sided relation is a tuple (a, b) and two factorizations:

$$(a - b\alpha) = \prod \mathfrak{p}_i^{\mu_i} \quad \text{and} \quad a - bm = \prod p_j^{\nu_j}$$

The smoothness bounds in these products vary as the descent tree grows, but they will reach B_{ext} and B_{rat} by the last step of descent.

On the surface, we reused sieving code for descent that the GNFS-DL would use in CADO-NFS. However a difference arises in parameter tradeoffs. In the GNFS-DL and SNFS-DL, the same smoothness bounds are used in descent and in precomputation. Given the massive amount of precomputation, the expected time to find a two-sided relation must remain quite low. The $\sqrt[3]{\text{NFS}}$ setting is different. Descent must find a few hundred two-sided relations, but one-sided sieving over its millions of relations can get away with a much more aggressive smoothness bound. This tradeoff was not discussed by JNT.

For a concrete number, in our 1024-bit computation, we selected $B_{\text{ext}} = 2^{35}$ and $B_{\text{rat}} = 2^{36}$. Each two-sided descent relation took several hours to find, while each one-sided precomputation relation took less than a second. We could have sped up descent by increasing B_{ext} to 2^{36} , but this would have doubled the amount of extension sieving. Descent took less than one month, while extension sieving took 2.5 months. The cost of doubling extension sieving was frightening.

(i) *Descent initialization.* Our descent algorithm takes place in three stages. The initialization stage borrows ideas from the 1024-bit SNFS-DL computation in 2016 [28]. It includes a `las`-plus-ECM step and a “large q ” step. In the former step, candidate initial splits $u/v = t \bmod N$ are found by sieving for candidate factorizations of u and v , while allowing for large (we chose 365-bit) cofactors. Those cofactors are handled by a more efficient ECM setup than is built into `las`. We aimed for 135-bit smoothness here. The “large q ” step handles the rational primes of size about 95 to 135 bits. They are too large to input to `las` as a special- q , so we used a technique that we call “embedded-polynomial sieving.” We describe it below in step (iii), where we used the same technique.

(ii) *Descent middle.* The middle of descent is its most tractable stage. We were able to use the existing `las_descent` code in CADO-NFS. For all of our computations, we required that this stage begin with special- q values of at most 95

bits. This is because the sieving lattice in `las` uses 64-bit integers, and has some dependence on the skewness of the polynomial.

On its own, this simple implementation of the descent middle was enough to handle moduli up to a 666-bit computation. However, we needed the additional initialization and bottom stages for the 768 and 1024-bit computations.

(iii) *Descent bottom.* To implement the bottom of descent, we used two strategies. The first is **composite sieving** and the second is **embedded-polynomial sieving**. During our large computations, we found that while both eventually find relations, composite sieving was usually faster. Composite sieving is simple to describe: rather than perform a single special- q (or q) job, it is possible to perform in parallel many special- (qq') jobs for small primes (or composites) q' . The benefit is parallelism. Concretely, we would run one `las` command, but with a long list of special- (qq') inputs on which to perform multi-threaded sieving. Composite sieving (albeit in a different form) was used in the computation from [14], which used `CADO-NFS`. It took very little work to adapt it to our setting.

Embedded-polynomial sieving is an idea from the “large q ” step and that we repurposed from [28]. Rather than find a special- q (or q) relation (a, b) under polynomials f, g it is possible find any relation (a', b') under transformed polynomials f_q, g_q such that (a', b') can be turned into a relation (a, b) under f, g which includes the prime q as a factor. The process is a homographic transformation. We briefly detail how that works, choosing as an example a special- q on the algebraic side: choose (a_0, b_0) and (a_1, b_1) two basis vectors of the lattice of (a, b) ’s such that q divides $(a - b\alpha)$. Here, the lattice is to be reduced according to the skewness parameter of the polynomial f . Let now $f_q = f(\frac{a_0x + a_1}{b_0x + b_1})(b_0x + b_1)^\delta$ (likewise for g_q , with δ the relevant degree). Notice that by construction, the skewness of f_q becomes ≈ 1 after this transformation. The benefits here are parallelism—since many small primes can be used to sieve under f_q, g_q —as well as injecting variety into the sieving lattices, in case a particular q has odd behavior. Although these ideas are not built into the `las` sieve, these bootstrapped (but folklore) uses of `las` were critical to making descent faster.

3.4 Detailed view: Linear algebra

The linear algebra step takes the rational function S output by descent and the information from precomputation, and it builds a matrix-vector equation where the solution yields rational functions U, T such that $S(\alpha)T(\alpha)U(\alpha) \in K$ is an eth power. That eth power is the input to the final root stage. While most of this step was handled by the `CADO-NFS` program `bwc` and extensive parameter optimization, we made two other nontrivial changes.

Linear algebra optimizations. In contrast to factoring, where the matrix is mod 2, and to discrete log, where the matrix is mod ℓ for a large factor ℓ of the group order, our implementation constructs a matrix that is mod the RSA public exponent e . We chose $e = 65537$ to match common use. Our matrix equation is inhomogeneous, like discrete log, and unlike factoring. These differences led

to extensive debugging of existing CADO-NFS linear algebra code. One interesting optimization we were able to use involves large matrix multiplications. The modulus $e = 65537$ is small enough that we could represent entries as `float64` values, and use optimized sparse matrix multiplication libraries [36, 65, 61] while keeping enough precision to reduce mod e at the end. This worked well up to our 768-bit computation, but the 1024-bit computation required more precision.

Filtering additions. Filtering is technically part of precomputation, but it involves the matrix we just discussed above. In this matrix, columns are prime ideals in $\mathcal{F}_{\text{alg}}$, rows are algebraic relations, and entries are valuations mod 65537. CADO-NFS includes a step called filtering which performs elementary operations on the matrix to make a smaller linear system. In this filtered matrix, the rows are actually linear combinations of the original relations, and the columns are a small subset of $\mathcal{F}_{\text{alg}}$. We used “killer relations” to map between filtered row or column vectors, to vectors over the full pre-filtered space. This is similar to a step called “reconstruction” in GNFS-DL but we needed our own implementation for this use case. The basic idea is to iteratively cancel ideals in a prescribed order, at each step using a so-called killer relation that includes only the current ideal and previously cancelled ideals. During our precomputation filtering phase, we thus built up this ordering of killer relations, and used it to appropriately apply or remove filtering on the linear algebra vectors.

3.5 Detailed view: eth root in a number field

Instead of using a single algorithm to find an eth root in a number field, we took a three-stage approach. This is the final step of $\sqrt[e]{\text{NFS}}$ before assembling the final forgery using Eq. (1).

(i) *Parallelized Montgomery root.* On the 768 and 1024-bit computations, we found that a coarse but more parallel version of Montgomery’s algorithm worked well for the first 90% of norm reduction. We followed the approach described in [68], but rather than selecting generators by analyzing the log-embeddings, we simply chose the first generator of the desired norm. We could choose up to a few dozen generators in parallel, and run only one accumulation step on the product of all the generators. Experimentally, this worked well in our computations. Moreover, it did not work well to try to pick generators intelligently (using the approach described below) in independent parallel instances; the log-embedding improvements tended to be cancelled out or even reversed. We believe there could be a more interesting way to exploit parallelism in Montgomery’s algorithm. This root algorithm can also be used (mod 2) in GNFS for factoring.

(ii) *Sequential Montgomery root.* This matches the approach described in [68]. Here, we ran a single thread to perform one reduction step at a time, selecting generators to balance the log-embeddings as much as possible. A purely sequential Montgomery root was fast for everything up to our 666-bit computation, but it was outperformed by the three-stage approach for 768 and 1024 bits.

(iii) *Hybrid CRT- p -adic root.* The Montgomery root, given a field element $R(\alpha)^e$, produces a list of γ_i such that $R(\alpha)^e = \delta^e \prod_i \gamma_i^e$ for some unknown δ with bounded coefficients; with each Montgomery iteration, the bound on the coefficients of δ (ideally) becomes smaller. One hopes that eventually δ will become 1, and $R(\alpha) = \prod_i \gamma_i$, but in practice this is not always the case. We then have to deal with the leftover δ^e , which we get as a product expression $\prod_i (a_i - b_i \alpha)^{m_i} \prod_j \gamma_j^{-e}$. The product is much too large to fully evaluate, but it can be evaluated mod some modulus p , as this prevents the coefficients from blowing up.

For the 768-bit computation and below, we handled the leftover case using the p -adic lifting approach described in [68]. In short, we choose an inert prime p (i.e. f is irreducible mod p) so that $\mathbb{F}_p[x]/(f(x)) \simeq \mathbb{F}_{p^{\deg f}}$, a finite field where e th roots can be taken easily. We evaluate the product expression for δ^e mod p^k (where p^k is above the coefficient bound on δ , but far below that of δ^e), take its e th root mod p (since e th roots in $\mathbb{F}_{p^{\deg f}}$ are easy), and then Hensel lift to recover the coefficients of δ mod p^k . Since p^k is above the coefficient bound, this recovers δ .

For our 1024-bit computation, we parallelized this step by using many inert primes p_i and solving for the coefficients of δ mod each p_i^k in parallel. We then recover δ via CRT reconstruction. (Here k can be smaller because we only need $\prod_i p_i^k$ to be above the bound, not each p_i^k .) This is thus a hybrid of the p -adic lifting and the CRT approaches of [68]. Compared to a purely-CRT approach, the hybrid approach let us use fewer primes (since we only require $\prod_i p_i^k$ be above the bound, not $\prod_i p_i$). As pointed out by Bernard, Fouque, and Lesavourey [12], unlike in the square root case, for odd e there is no need to guess the correct signs of the e th roots mod each p_i before CRT reconstruction: e th roots are unique in $\mathbb{F}_{p_i^{\deg f}}$ as long as e is relatively prime to $p_i^{\deg f} - 1$.

The Montgomery root algorithm is asymptotically faster than CRT and/or p -adic, but the latter is much more parallelizable. It seems likely that the root step could be made faster by carefully balancing the two.

3.6 Detailed view: Parameter selection

To the frustration of the reader and any interested replicators, our parameters were chosen through a combination of intuition and small-scale experiments. For many parameters, it was infeasible to exhaustively run experiments on every combination of allowed parameters on academic resources. Our process was to first decide on high-level choices ($B_{\text{rat}}, B_{\text{ext}}, B_{\text{alg}}$) then run experiments with mid-level choices (A, mfb , etc.). For sieving, our experiments tended to optimize the number of unique relations found per second. For descent sieving, we also accounted for RAM usage; we increased A, mfb until parallelized sieving ran out of memory on our 512GB machines. For the Montgomery root, our experiments increased the per-iteration norm bound until we either ran out of precision in our called `SageMath` and `PARI` code, or ran out of memory.

Regarding the smoothness bounds, B_{alg} was always selected first, and not changed afterwards. For some of the computations, we ran descent and extension

sieving on different subsets of machines, so that we could compare their expected finishing times and adjust as needed. In particular, descent sieves from top (large primes) to bottom (smaller primes). We ran extension sieving starting from B_{alg} (small primes) and going up to B_{ext} (larger primes). Thus both bounds $B_{\text{ext}}, B_{\text{rat}}$ could be increased without losing past work.

We ran fewer experiments for linear algebra but were informed by past NFS computations. We chose to follow [28] which selected a matrix of primes bounded by 2^{31} for a 1024-bit SNFS-DL computation. This was successfully run for a much larger matrix modulus, so we expected to slightly outperform their timing. We ended up taking 19 days; theirs took one month.

4 1024-bit computation details

The public RSA key on our HSM consisted of:

$$e = 65537$$

$$N = \text{aa8bb728b43ba939dc282ae5eace9e939ae5a985398db5816d4e266a1a64986d} \\ \text{21a19152fb3cb738aa7ecd3b868f5ed42d6afd2ce33d450ab68ff0d8705604e4} \\ \text{72d8a235a06b743bb5ea2b731585cafe38d9b5ec6f0f1aefd86dc26426891144} \\ \text{67bb7ede0a3314daed0f3d184a7f605105ebd49dee447bbc23e331dfa8e1c113}$$

with $\log_2(N) = 1024$. The random target that was selected for forgery, and the eth root that we ultimately computed, were:

$$t = \text{804fbc7eb701fdc8fa1147900eb44395ace53ac96a9c676726131ef4fb4158fa} \\ \text{33c50e3bf4bda303d8a6a8c51c0a3031b6a0c12378d7eb3b760ad760c30790af} \\ \text{d64517920348c12e26ed75f233a4e731f772309c5df0873d8bcab2c116e3ff04} \\ \text{3ea1ee3379eac327c58686b40f87edae312b0534a50df24206caa1e220368eb8} \\ t^{1/e} = \text{944678c4efa56b54dcaff74fac3cca0acc5dd571d18af653171693d8560fc2cf} \\ \text{752e62edc5ba2a6f378e7afdebf3d3ffb3af6e1c139764d3753e9976da2dad12} \\ \text{aa94e8365ae7cb599fdc528bc0618fc789fc0a27aac8403e101d4611c5faff62} \\ \text{5832ca3c3509a7f021553d49e434dade3a097b0adfed94468a1d5e1d9ee68f88}$$

The target was selected at random from $\mathbb{Z}_N^*$. It can be easily verified that $(t^{1/e})^e \bmod N = t$.

Our computation ran from March 13 to August 31, 2026. Due to breaks and (un)expected cluster downtime, the actual running time was about 5 months. We chose to run certain steps in parallel to more quickly identify problems that arise only for very large moduli; there were such problems at nearly every step. If running only one step at a time on 100% of the cluster, the breakdown of the 5 months would be about: 2.5 months of extension sieving, 0.5 months of algebraic sieving, 1 month of descent, and 1 month of linear algebra and the final steps.

Step	CPU time (h)	Wall time (h)	Notes
Precomputation			1200 core-years
Polynomial selection	39,992.01	20.17	Small portion of cluster.
Algebraic sieving	1,311,975.08	2,760.00	
Filtering*	24.08	7.85	
Extension sieving	9,240,757.82	2,640.00	Ran on $> 1/2$ of cluster.
HSM queries			4.1 billion queries
Rational queries*	N/A	123.12	2,874,398,515 queries
Algebraic queries*	N/A	6.59	144,973,282 queries
Extension queries*	N/A	45.15	1,048,047,534 queries
Individual forgery			180 core-years
Descent (i)	17,141.54	19.82	Ran on $< 1/2$ of cluster; each core used significant RAM.
Descent (ii)	18,811.00	268.36	
Descent (iii)	1,385,242.92	739.21	
Linear algebra*	146,263.96	447.04	
Root*	2,210.93	147.16	

Fig. 4. Timing of our 1024-bit computation. All steps other than those marked with a (*) can be trivially parallelized over more machines, meaning the wall-clock time can be easily decreased with more resources.

4.1 Timing, memory usage, and queries

The timing of each step is summarized in Figure 4. The CPU time should be viewed as additive. The wall-clock time is not, due to how we scheduled steps to be run on different subsets of the cluster. Some steps of the algorithm are trivially parallelizable, meaning that if twice as many machines were used, the wall-clock time would be cut in half. This applies to all steps except those marked with a (*) in Figure 4; notably, linear algebra and the Montgomery root.

Our “cluster” included a few different types of machines. We used:

Cluster A : 45 machines with 48–88 cores and 256GB–1TB of RAM

Cluster B : 16 machines with 256 cores and 1TB of RAM

Cluster C : (Varying) machines with 32+ cores and 192GB+ of RAM

Cluster D : 2 machines with 512 cores and 1TB+ of RAM

The above machines have hyperthreading enabled and we report virtual cores; there are half as many physical cores. The bulk of the work was done on **Cluster A** which has relatively dated processors from 2016 but we had uninterrupted access to all machines. We had priority access to **Cluster B** and some access to **Cluster C**, where we mostly performed extension sieving. We had short-term access to **Cluster D** where we performed some extension sieving, as well as high-RAM steps including the Montgomery root.

4.2 Parameters selected

The global smoothness bounds we selected were:

$$B_{\text{rat}} = 2^{36} \quad B_{\text{ext}} = 2^{35} \quad B_{\text{alg}} = 2^{31}.$$

These are the three most consequential to the running time. For our polynomials, we selected $\deg(g) = 1$ as always for a rational side, and $\deg(f) = 6$.

Sieving occurs in three different steps in the $\sqrt[n]{\text{NFS}}$ algorithm, and they all used different sieving parameters. Ordered from easiest to hardest to manage, they are: algebraic sieving, extension sieving, then descent sieving.

The linear algebra and root parameters described below were determined largely from the above, as well as by the exponent being $e = 65537$.

By this point, the reader may notice that we counterintuitively set $B_{\text{rat}} > B_{\text{ext}}$. In usual NFS computations with a rational side, one would prefer to set $B_{\text{rat}} < B_{\text{ext}}$. Here, recall that B_{ext} is the smoothness bound on the algebraic side, when two-sided sieving is performed. For example, in the 2019 RSA-240 factorization via GNFS [14], $B_{\text{rat}} = 2^{36}$ was used, and the equivalent of B_{ext} was set to $B_{\text{ext}} = 2^{37}$. Roughly, this is because the algebraic side is “heavier” meaning that for a given relation, the algebraic side tends to have a larger norm, which is less likely to factor into small primes. We did observe the same behavior in two-sided sieving. However, the $\sqrt[n]{\text{NFS}}$ algorithm has a different global tradeoff between B_{rat} and B_{ext} . To make two-sided sieving in descent tractable, one can either increase B_{ext} , which roughly doubles the cost of extension sieving in precomputation and doubles the number of extension queries; or one can increase B_{rat} , which incurs *no extra sieving cost* but still doubles the number of rational queries. We made the second choice more often than the first, because we were more concerned with sieving time than with query time.

4.3 Possible tradeoffs and alternative regimes

We chose the above parameters with the goal of making the end-to-end computation work in a handful of months. Different high-level parameters could be selected to optimize other goals. The point of this discussion is not to provide definitive answers about alternative parameter regimes (which would need to actually be run) but to suggest that ours is not the only one possible.

For example, another realistic goal is to minimize the number of HSM queries, especially if the attacker has limited online access to the HSM. In this paradigm, one could feasibly select $B_{\text{rat}} = 2^{34}$ and $B_{\text{ext}} = 2^{34}$. Our experiments suggest that this would cut the number of queries to 1.5 billion while significantly increasing the time spent in descent (perhaps by a month).

On the other extreme, a goal could be to minimize the time spent in individual forgery. We believe it would be hard to decrease the time spent in linear algebra, but descent could be sped up by increasing B_{rat} and B_{ext} . Our experiments suggest that the individual forgery could be made to take 5–6 weeks, but this starts to require tens of billions of queries.

4.4 Trip report: Precomputation

Polynomial selection. We used a similar polynomial selection approach to prior factorization computations. After using **CADO-NFS** scripts to generate candidates and perform root optimization, we narrowed our search to the 100 best polynomials based on the “Murphy E” value.

Given these 100 candidates, we performed about 8 hours of sieving experiments on our cluster, testing the polynomials with different sieving parameters. Ranking by the average time to find a unique relation, there was a clear winner and runner-up. We also verified that these polynomials satisfied necessary properties for our root computation to work: first that $e = 65537$ was unramified in the corresponding number field; and second that there were several small inert primes with which we could perform p -adic lifting.

A last consideration was skewness. Because descent involves sieving where the special- q or special- $\mathfrak{q}$ can be up to 95 bits, we needed to be careful with the size of entries in the sieving lattice, which in **CADO-NFS** are represented as 64-bit unsigned integers. Since the larger entries are of size about $\sqrt{q} \cdot \text{skewness}$, we chose to aim for a skewness of about 2^{16} . This was reflected in our input parameters to the **CADO-NFS** candidate-generation program. The polynomials we selected are:

$$\begin{aligned}
 g(x) = & + 583567398878296587761x \\
 & - 583508354875246158635574786673190632633220109293 \\
 f(x) = & - 3823002681315339817451520x^6 \\
 & - 7350594038360643424557871006920x^5 \\
 & - 216020850394775040895128014268735128x^4 \\
 & + 1766821832922784779107212285802811346027985x^3 \\
 & - 10216745863874085463847140035876784402919224735x^2 \\
 & - 29161682848128620601357534837213030324970598132169770x \\
 & - 1140093999763033664163418590780932917435250519974808516832
 \end{aligned}$$

Here, g is the rational-side polynomial, f is the algebraic-side polynomial, and $g(m) = f(m) = 0 \pmod{N}$ for a shared root m . The relevant number field is $K = \mathbb{Q}(\alpha)$ where $f(\alpha) = 0$. The skewness is 267, 925.

Algebraic sieving. Algebraic sieving was the most straightforward step of our computation. Given our polynomial f , we used **CADO-NFS** to create a factor base of all prime ideals of $\mathcal{O}_K$ with norm at most $B_{\text{alg}} = 2^{31}$. Then we used **las** to sieve over large ranges of special- $\mathfrak{q}$ until we had at least one relation covering every $\mathfrak{q}$ in the factor base.

We used a sieving area of $A = 29$ and increased to $A = 30$ as we needed more relations. We used a cofactor of $\mathbf{mfb} = 100$ and increased to $\mathbf{mfb} = 120$; in other

words, we allowed for 3–4 large primes. We actually used a smaller factor base limit of $\text{lim} = 2^{27}$ since we found that this smaller limit (with a larger mfb) was faster on our machines. **CADO-NFS** allows for multi-layer bucket sieving, which we did use (see [14, §4.2]). We set $\text{bkthresh1} = 2^{26}$. Recall that this is all *one-sided* sieving, which explains the perhaps smaller-than-expected parameters.

Filtering (which we detail below) is a step that processes the found relations into a matrix. After filtering, it turned out that some of our factor-base ideals became unlinked, due to singleton elimination or other simplifications. For these ideals, we performed special- $\mathbf{q}$ sieving to find a few more relations for each $\mathbf{q}$. Rather than increasing A and mfb , we found that it was more helpful to perform composite sieving over $\mathbf{q} \cdot \mathbf{q}'$ for small prime $\mathbf{q}'$. While some unlinked ideals may not be a problem for the GNFS, for our setting, we wanted to cover 100% of ideals so that any descent output could give us a solvable linear algebra equation.

Extension sieving. Although extension sieving was by far the most computationally expensive step in our computation, it was luckily predictable and consistent to run. Recall that here, we are sieving on one side with special- $\mathbf{q}$ that are between $B_{\text{alg}} = 2^{31}$ and $B_{\text{ext}} = 2^{35}$, with smoothness bound $B_{\text{alg}} = 2^{31}$. We used the same parameters as above for algebraic sieving, but started out at the higher end of $A = 30$ and $\text{mfb} = 120$.

Other than the scale and sizes, there were two key differences in extension sieving compared to algebraic sieving. First, the goal is to find only *one* relation per target ideal. Although we set the **CADO-NFS** parameter to “exit early” we also needed to de-duplicate afterwards, since this flag does not fully stop all threads. It is a different de-duplication step than the one used for algebraic sieving.

Second, we did not need nor want 100% coverage of all the ideals between $B_{\text{alg}} = 2^{31}$ and $B_{\text{ext}} = 2^{35}$. On average, relations are straightforward to find, but the tail end of hard-to-include ideals quickly becomes an expensive task. We aimed for 90% coverage. Then, descent jobs would be likely to include our linked ideals, and for descent relations involving the 10% of unlinked ideals, it was only a little more work to find a second relation avoiding these. The coverage of ideals that we ultimately found was all prime ideals whose norm was below 2^{31} , 90% below 2^{33} , and down to 60% below 2^{35} . Practically, this struck a reasonable balance with descent. Overall, however, extension sieving is a unique trick to the $\sqrt[N]{\text{NFS}}$ algorithm, and we believe other tradeoffs could be justified.

Filtering and matrix preparation. The final step of precomputation is to prepare the matrix for linear algebra. We used standard **CADO-NFS** steps, plus an additional step to account for what we called “killer relations.” We used all of the **CADO-NFS** filtering programs (**dup1**, **dup2**, **purge**, **merge-dl**, **replay-dl**) with discrete-log flags, since our linear algebra setup closely matches that of GNFS-DL rather than GNFS. The only nontrivial parameters we set were to aim for a target density of 210, and to keep at least 100 extra rows after **purge**. The sparse part of our filtered matrix had 10,557,419 rows and 10,557,319 columns, among which there were 2,217,211,872 nonzero entries. Each row had an additional 6 dense righthand columns for the Schirokauer maps.

4.5 Trip report: Oracle queries

To instantiate the RSA signature oracle, we first had to obtain access to an HSM. Enterprise-grade HSMs are expensive proprietary cryptographic accelerators that are opaque by design. They are usually sold exclusively on the business-to-business (B2B) market and not to end consumers. The smaller, consumer-grade HSMs that we considered typically disallowed 1024-bit RSA. We gained access to the following two models, through different means:

- *Thales Luna K6 (PCIe 7000)*: This HSM is an expansion card that can be plugged into a server’s PCIe slot. It was donated to us and had an estimated market price in the high four digits or low five digits⁶ before being deprecated in June 2021 (for companies) and December 2023 (for governments) [50].
- *Thales Luna S750*: A third party allowed us to use their HSM remotely. It is still sold for \$22,000–32,000 US dollars [35, 40] and does not appear to have an end-of-life date yet.

In addition to the above HSMs, we unsuccessfully attempted to set up several other HSMs locally but were unable to find a copy of the proprietary software that is required to set them up. In the following, we first describe how we configured our local Luna K6 HSM, followed by our system architecture for querying both the local and remote HSM. To conclude, we report the wall-clock time for querying, as well as the signature speed of the two HSMs.

Setting up the Luna K6 HSM. After installing the Luna K6 HSM in a slot that is (backwards) compatible with PCIe x4, we had to install the necessary software to interact with the HSM. Together with the Luna K6 HSM, we received a CD-ROM with proprietary software for this HSM model. However, due to changes to the Linux kernel headers and other dependencies, we discovered that Ubuntu 10.04 is the latest OS for which this software can be compiled. We could not emulate this old Ubuntu version since the HSM registers as a PCIe bridge, which cannot be passed through to a VM. Hence, we did a bare metal install, which meant we had to manually backport several packages, for example to support the network card of our server which was produced after Ubuntu 10.04 was declared end-of-life. To interface with the HSM we installed the pycryptoki [62] PKCS#11 wrapper for Python developed by the Thales.

For our local HSM, we had to obtain a PIN Entry Device (PED) [60] and PED keys [59] from the second-hand market. The former is an external peripheral device with a small display, numeric keypad, and a few action keys. It serves as a trusted input/output device which bypasses the untrusted server where the HSM is plugged into a PCIe slot. The PED is used during setup time to program PED keys—which are programmable devices of a similar form factor as USB keys—with key material for various privileged roles. We further enabled “user activation” and “auto-activation”, which generate a secret value displayed on the PED that can later be used to start user sessions on the HSM from remote.

⁶ We could not find any concrete prices listed on the public internet since this HSM was exclusively sold on the B2B market.

In the global HSM settings, we disabled FIPS 140-2 approved operation mode to enable raw RSA operations (referred to as `CKM_RSA_X_509` in the Cryptographic Token Interface Standard [20]). Querying unpadded messages is necessary for our attack.⁷ Afterwards, we create a partition which inherits the global HSM settings. We chose to do key generation outside of the HSM and then import it because HSMs wipe their state on a power outage (of which we had multiple during this project). For this, we generate a 1024-bit RSA key in PKCS#1 format and store it in DER encoding and imported it with the Certificate Management Utility (CMU) from the Luna client software. HSM deployments would instead likely use the cloning or wrapping features, through which the keys can be transferred directly to another HSM of the same model without the keys ever leaving the HSM unencrypted.

Query architecture. Our goal was to have a joint system that runs queries both on our Luna K6 HSM and the remote Luna S750. For the latter, we were not given access to the third party’s network but were only allowed to pass software to them. Hence, we designed the following three-piece system architecture. First, a *local query handler* groups queries into batches and uploads them to the web server. It also regularly attempts to download new signature batches and aggregates them. Second, the *web server* offers an authenticated REST API to upload and download batches, metadata, performance metrics, and debug information. Third, the *remote query handler* fetches query batches from the web server, runs them on the HSM, writes the resulting signatures into batches, and uploads them.

Various code handles network and hardware failures, performs rate-limiting, and re-synchronizes the handlers. Both query handlers send measurements of their upload and download bandwidth to the web server. Additionally, the remote query handler uploads the measured signature signing speed, as well as CPU and memory usage to the web server. All components are highly parallelized. We carefully chose the batch sizes, number of batches in transit, number of threads to write, download, pre or post-process batches, and webserver parallelism such that the bottleneck was at the HSM, not the network bandwidth or batch pipeline. The remote query handler catches most exceptions, reports them to the web server to enable remote debugging, and then restarts gracefully.

Query timing. The HSM does not use parallelism within sessions, even under a high query load. Instead of multiple OS threads using the same session (which throws exceptions on the HSM), a thread per session should be used to benefit from parallelism and maximize the signature speed. The official administration guide recommends using 20–40 threads [30]; we achieved an average of 6500 signatures per second with 40 threads. Computing the first batch of rational queries on our Luna K6 HSM took over a month due to many pauses for debugging corner cases that only occurred at scale (e.g. unlikely concurrency bugs).

⁷ Applications may also choose to enable raw RSA operations to apply a padding scheme that is not implemented inside the HSM as we discuss in Section 7.1.

Based on the average signature speed, running the rational queries would have taken 123.12 hours without interruptions. The algebraic and extension queries took 6.59 and 45.15 hours respectively, showing that the HSM ran at impressively consistent signature speed. The overhead from our query architecture was overall minor and stemmed from ramping up the pipeline utilization, error handling and re-synchronization in case a garbled batch arrived, and post-processing for the very last batches.

Additionally, we ran the same queries on the remote Luna S750 HSM, which was located on a different continent. We initially expected the Luna S750 HSM to be faster than ours, since it is a newer generation, but that was not the case. The rational, algebraic, and extension queries took 317.25, 7.08, and 118.93 hours, respectively. This corresponds to a signature speed of approximately 2500 sig/s for the rational and extension queries, while the algebraic queries ran at almost 5700 sig/s. The overall slowdown is not due to our query architecture or network latency but more likely because we could not optimize the parameters without direct access to the HSM. Indeed, we asked them to increase the number of threads for the algebraic queries, which more than doubled the signature speed.

Using multiple HSMs as query oracles shows that our attack is mostly platform-agnostic since it only makes black-box use of the HSM functionality for RSA signatures. Even currently used HSMs can be in a configuration state that allows us to run our attack. Furthermore, the Luna S750 HSM test shows that it is possible for an adversary to run these queries over the internet. As shown in Figure 4, we made a total of 4,067,419,331 queries.

4.6 Trip report: Individual signature forgery

The three phases of the individual forgery (descent, linear algebra, and the ϵ th root) all differed in some substantial ways from usual NFS cases. To start off, we generated a random target $t \leftarrow \mathbb{Z}_N^*$.

Descent (i). The first step is descent, which, again, can be broken down into several substeps. The algorithmic ideas are described in Section 3; here we present the progression of the actual 1024-bit computation. Recall from Section 3 that we had four strategies to tackle descent, each specialized to slightly different phases. These were initialization (which closely followed the ideas of [28], and which we can more descriptively call `las-plus-ECM`); the `CADO-NFS GNFS-DL` program `las_descent`; composite sieving; and embedded-polynomial sieving. These last two were the main new ideas we applied to descent, as they have previously not been used in the middle and bottom stages. In part, this is because for our chosen smoothness bounds, the difficulty of descent (compared to one-sided precomputation) is unique to the $\sqrt[N]{N}$ algorithm.

First, to initialize descent, we greatly benefitted from launching hundreds of jobs, then picking the smoothest u/v split we saw. We spent a day running 500 initializations in parallel, each using a different mask and sieving to find $t \cdot \text{mask}^e = u/v \bmod N$. We used smoothness parameters `lpb0` = `lpb1` = 135, very large cofactor bounds `mfb0` = `mfb1` = 365, sieving area $A = 31$, and factor

base bounds $\text{lim0} = \text{lim1} = 2^{31}$. To perform the large ECM cofactorizations we followed the strategies and scripts of [28]; we printed surviving relations from `las` to files, on which we ran multithreaded ECM jobs.

After this day of sieving and ECM, we had a clear best initial split: u/v factored over primes bounded by 80 bits. In fact, we were only aiming for 135 bits (in the `lpb` parameters above), but took this lucky split. Because of this, we skipped a step we had needed for the 768-bit computation: the “large q ” step we described in Section 3 which performs the jump from 135 to 95 bits. Thus initialization brought us all the way down to 80 bits.⁸

Descent (ii)-(iii). The remainder of descent brought us down from 80 bits (on the rational side) to our final smoothness bounds of $B_{\text{rat}} = 2^{36}$ and $B_{\text{ext}} = 2^{35}$. In brief, we used the following strategies:

- For most primes from 80 bits to 40 bits: `las_descent`
- For straggler primes from 80 bits to 40 bits: composite sieving or embedded-polynomial sieving, with a few hundred special- q or special- q values
- For primes from 40 bits to the end: composite sieving, with thousands of special- q or special- q values

A strategy which did *not* work well was simply running `las_descent` with larger and larger A . Even on the machines that could allocate up to $A = 39$, sieving was outperformed by the more parallelizable composite sieving with smaller A .

The parameters we used for the above three strategies were closely tuned to the machines we used; our goal was to use most of the available RAM while maintaining reasonable multi-threading. The `memory_margin` parameter was also critical to ensure we didn’t over-allocate memory. For `las_descent` we used $A = 37$, $\text{lim} = 2^{31}$, $\text{bkthresh1} = 2^{29}$, and $\text{bkmult} = 1.477$. For composite sieving at the bottom, we decreased all these parameters. We used $A = 34$, $\text{lim0} = 2^{28}$, $\text{lim1} = 2^{30}$, $\text{bkthresh1} = 2^{27}$, and $\text{bkmult} = 1.365$.

A final reflection on this 1024-bit descent computation is that although composite sieving worked effectively in practice, it was difficult to predict its success theoretically. Recall that here, when we have a target q to cancel in descent, we sieve over special- q values of q , $q \cdot 11$, $q \cdot 13$, etc. with small prime (or composite) multipliers. These sieves are not independent, but in practice, certain special- q values seemed to have an easier time in `las`. Even if one of the first ≈ 150 primes showed up in a smooth relation, that relation was sometimes only found when a larger prime multiplier was used. All primes other than the last eight were found (meaning: we found a smooth relation involving that prime) with up to 10,000 small multipliers; most were found early on. For those last eight—which were

⁸ Suppose we were less fortunate and had to run a “large q ” step. Would this significantly change the timing? For our 768-bit computation, the answer was no. It took 98 core-hours compared to 10,000 core-hours spent in the `las`-plus-ECM step. A general challenge with descent was the fact that the computation must work on one particular target. Although smoothness and sieving are all predicated on average behavior, we certainly encountered lucky, unlucky, and average behavior.

Step	CPU time (h)	Wall time (h)	Parallelism
Linear algebra			
Matrix/vector preparation	530.59	11.52	1 machine, 88 threads
<code>bwc_krylov</code>	74,579.78	208.88	10 machines, 88 threads each
<code>bwc_lingen</code>	75.96	1.72	
<code>bwc_mksol</code>	71,073.63	221.42	
Second (character) solve	0.47	0.47	1 thread
Cleanup/sanity checks	3.54	3.04	1 thread
eth root			
Parallel Montgomery	1,305.79	130.58	1 machine, 10 threads
Sequential Montgomery	1.15	1.15	1 thread
CRT- p -adic root	896.20	11.09	1 machine, 88 threads
Forgery assembly	7.80	4.35	1 machine, 88 threads

Fig. 5. Detailed timing breakdown for linear algebra and the *eth* root.

not remarkable in other ways—we went up to 20,000 small multipliers. Although this tail end of descent was small, it did affect the wall time of descent.

Our ultimate descent tree included 5,633 rational primes bounded by B_{rat} , 8,592 prime ideals bounded by B_{alg} , and 1,860 prime ideals bounded by B_{ext} . These were spread over 874 relations.

Linear algebra. We give a further breakdown of the linear algebra step in Figure 5. Our first observation is that the CADO-NFS `bwc` program—after some debugging and fixes—handled our linear algebra system quite well. In part, this is because linear algebra in $\sqrt[e]{N}$ FS is an easier task than in GNFS-DL. The modulus $e = 65537$ is very small, the factor base of primes (columns) includes only one side, and thanks to extension sieving, we could use an eminently reasonable smoothness bound $B_{\text{alg}} = 2^{31}$. On the other hand, `SageMath` and Python stopped working well (if at all) on matrices of this size. In the 1024-bit run, we omitted several assertions and multiplications since they ran out of memory on our 1TB machine. It seems difficult, in general, to handle a large, sparse, modular matrix.

Despite this, we modified the critical code paths to work. The most important parameter was `bwc_mn` = 10, which determines how many solutions are found by `bwc` and how some steps are parallelized. We performed two linear algebra runs: one with additional parallelism via MPI [56], and one without it. The latter instance ran for weeks without interruption and found a correct solution. This is the linear algebra step reported in our timings. The MPI run required substantial manual intervention, but not counting the idle time during breaks, it found a correct answer in shorter wall time but slightly larger CPU time. Concretely, the MPI run used roughly twice the number of machines compared to the non-MPI

run, and had the following timings for the two most compute-intensive steps of linear algebra:

- `bwc_krylov`: 180.73 h (wall time); 80,999.67 h (CPU time); speedup: 1.16x.
- `bwc_mksol`: 179.56 h (wall time); 75,930.18 h (CPU time); speedup: 1.23x.

The other steps had runtimes comparable to the reported non-MPI run. We note that MPI was relatively far away from achieving the maximal possible speedup of 2x in this case. Using additional MPI nodes for every task would further reduce the wall clock time, but eventually see diminishing results due to the communication overhead. Our choice of the number of MPI nodes was due to resource limitations. MPI would likely be more beneficial for larger matrices.

eth root. Timings of the different root stages are in Figure 5 while the algorithms were explained in Section 3. The parallel Montgomery step could not fully utilize the cores on our machine due to high memory usage, as well as limits to independence in reduction steps. It was also hampered by a few `SageMath` inefficiencies.

At the start of the root step, our numerator and denominator each had about 209 million bits. By the end of the Montgomery step, we had cleared the denominator, and the numerator was bounded, for each coefficient of the number field element, by about 10^{2480} . To get to the very end, our CRT- p -adic step worked well and is included in our timing reports.

We ran a few other tests, not included in our timing. A purely p -adic instance (after Montgomery) with $p = 70111$ found a solution after lifting up to 70111^{256} in about 81 hours of wall time. We ran another Montgomery step with different parameters, and reached an improved numerator bound of 10^{468} . We certainly believe there is room for optimization in the root step, but were satisfied with the reported results.

5 What we talk about when we talk about RSA

The JNT algorithm has been public for two decades, and the research community has been investigating the relationship between RSA and factoring for several more decades. In this section we discuss some consequences (and non-consequences) of $\sqrt[e]{NFS}$ becoming feasible to run on large moduli.

Why doesn't $\sqrt[e]{NFS}$ break RSA? To start off, we will continue to emphasize the difference between RSA and delayed-target RSA. Although JNT's paper is titled "When *eth* roots become easier than factoring" it is more accurately "When *eth* roots with an oracle become easier than *eth* roots without an oracle." In the standard RSA problem, the goal of an attacker, given a public key (N, e) and target t , is to output $t^{1/e} \bmod N$. Without the oracle, $\sqrt[e]{NFS}$ cannot be run as is. Even if one could run descent and the rest of the individual forgery phase, it is not obvious how the forgery could be assembled without all the *eth* roots (coming from oracle queries) on the forgery components in Eq. (1).

One idea is to collect signatures in the wild: for a particular (N, e) , could you observe billions of public signatures to replace the oracle queries? In practice, valid signed messages are hashes that are padded to the length of the modulus (albeit often with fixed-pattern padding as in PKCS#1v1.5). These messages will never correspond to the exact rational primes, and rarely to the smooth relations found during precomputation. One might hope to occasionally observe signed values that are smooth with respect to the factor base, and after observing enough such signed values, solve for the signatures on each of the factor base primes. However, this is likely to be much more expensive than the GNFS without additional cleverness. Such a strategy is asking for many full-sized integers to be smooth, which occurs with much smaller probability than the integers sieved over during the GNFS.

Why doesn't $\sqrt[e]{NFS}$ break factoring? Another observation is the following. The $\sqrt[e]{NFS}$ algorithm allows someone to, after the precomputation, forge *any* RSA signature of choice. From the outside—and completely ignoring timing—it seems that the attacker must know the secret key, or at least as much information as the secret key.

We do not have any formal arguments, but have two intuitive reasons that $\sqrt[e]{NFS}$ is different from factoring. First, recall that factoring N is equivalent to computing square roots mod N . Even if one had the power to compute arbitrary odd roots mod N ; say, $t_1^{1/3}$, $t_2^{1/5}$, $t_3^{1/7}$, and so on for any targets of choice; it is not clear how they could be efficiently combined into an inverse even power.

A second observation we found interesting is that technically, $\sqrt[e]{NFS}$ is only faster than GNFS if the goal is to forge a reasonable number of target signatures. If the goal is to forge subexponentially many signatures for a particular N , for example, it is likely more efficient to run GNFS and learn d than to run the descent, linear algebra, and root steps subexponentially many times. Although we cannot imagine a scenario where this would make sense, there is certainly a range of possible attack strategies.

5.1 One-more and delayed-target RSA

Finally, we explain why we have called our setting the delayed-target RSA problem (DT-RSA), rather than the more popular one-more RSA problem (OM-RSA) [8]. Indeed, it was claimed in [42, §4] that the $\sqrt[e]{NFS}$ breaks a version of the OM-RSA problem. With a cursory analysis, however, this is misleading.

It is true that $\sqrt[e]{NFS}$ solves the DT-RSA problem, and it is true that a polynomial-time attack for DT-RSA yields one for OM-RSA. It is even true that a subexponential-time attack for DT-RSA yields one for OM-RSA. However, when the obvious modifications are applied to $\sqrt[e]{NFS}$ to work in the OM-RSA game, its subexponential parameters are so much worse that they are outperformed by GNFS factoring.

The high-level takeaway of this distinction is that in spirit, $\sqrt[e]{NFS}$ does not attack a one-more problem; it attacks a CCA1-style problem. It is certainly possible that there is an $\sqrt[e]{NFS}$ -inspired algorithm that more efficiently attacks

Game $\mathbf{G}_{\mathbf{Kg}, \ell}^{\text{dt-rsa}}$	Game $\mathbf{G}_{\mathbf{Kg}, \ell, r}^{\text{om-rsa}}$
INIT: 1 $(N, e, d) \leftarrow \mathbf{Kg}$; done $\leftarrow$ false 2 For $i = 1.. \ell$ do $t_i \leftarrow \mathbb{Z}_N^*$ 3 Return (N, e) GETTGT: 4 done $\leftarrow$ true ; Return $(t_1, \dots, t_\ell)$ GETQUERY(w): 5 Require: done = false 6 Return $(w^d \bmod N)$ FIN($z_1, \dots, z_\ell$): 7 Return $((z_i^e = t_i \bmod N) \forall i \in [1, \ell])$	INIT: 1 $(N, e, d) \leftarrow \mathbf{Kg}$ 2 For $i = 1.. \ell$ do $t_i \leftarrow \mathbb{Z}_N^*$ 3 ctr $\leftarrow$ 0 4 Return $(N, e, t_1, \dots, t_\ell)$ GETQUERY(w): 5 ctr $\leftarrow$ ctr + 1 6 Return $(w^d \bmod N)$ FIN($z_1, \dots, z_\ell$): 7 Require: ctr = r 8 Return $((z_i^e = t_i \bmod N) \forall i \in [1, \ell])$

Fig. 6. Oracle RSA problems, given ℓ targets. We **Require** conditions to rule out trivial wins. Technically, OM-RSA is the “chosen-target inversion problem” of BNPS [8].

OM-RSA, and we do not claim otherwise. The fact that blind signatures tend to rely on the OM-RSA security assumption is interesting. The $\sqrt[\ell]{\text{NFS}}$ algorithm seems to be an eminently relevant and practical attack, but its speedup over factoring is not captured (without modification) by the OM-RSA problem.

An attack on DT-RSA yields an attack on OM-RSA. Formal game-based definitions for the DT-RSA and OM-RSA problems are given in Figure 6. The games are parameterized by key-generation algorithm $\mathbf{Kg}$ which for RSA is required to output (N, e, d) such that $N = pq$ is a biprime and $ed = 1 \bmod \varphi(N)$. They are also parameterized by a number ℓ of targets, and for OM-RSA, a number r of queries. In the below proposition we prove that an attack on DT-RSA yields one on OM-RSA.

The proposition is not meant to be exhaustively formalized; it is intended to be simple and to illustrate a gap while abstracting the details of $\sqrt[\ell]{\text{NFS}}$. This may lead to better future ideas.⁹

Proposition 1. *Let $\mathbf{Kg}$ be an RSA key-generation algorithm and let $\ell, r \in \mathbb{N}$. Given an adversary B in game $\mathbf{G}_{\mathbf{Kg}, \ell}^{\text{dt-rsa}}$ making r queries we can build an adversary A in game $\mathbf{G}_{\mathbf{Kg}, \ell, r}^{\text{om-rsa}}$ with*

$$\mathbf{Adv}_{\mathbf{Kg}, \ell}^{\text{dt-rsa}}(B) \leq \mathbf{Adv}_{\mathbf{Kg}, \ell, r}^{\text{om-rsa}}(A). \quad (2)$$

Adversary A also makes r queries. Its running time is about that of B .

Proof. The proof is straightforward. Given adversary B for DT-RSA, we construct adversary A for OM-RSA. A receives as input $(N, e, t_1, \dots, t_\ell)$. It runs B

⁹ Nonetheless, our best future idea is to retire RSA.

on inputs N and e ; the queries to B 's supposed GETQUERY oracle are answered by A forwarding the same inputs to and outputs from its own oracle.

When B finishes precomputation and makes its GETTGT query, A returns targets $(t_1, \dots, t_\ell)$. When B finally submits its ℓ forgeries, A returns the same ones. These are all correct e th roots as long as B was correct. If B made r queries to GETQUERY, then so did A . The running time of A is about the same as B . $\square$

Why doesn't $\sqrt[\ell]{\text{NFS}}$ break OM-RSA? We can use the above proposition to build an attack A on OM-RSA given the attack $\sqrt[\ell]{\text{NFS}}$ on DT-RSA. The main difficulty will be the parameter r , the number of queries.

Throughout this whole paper, we have implicitly assumed the number of targets ℓ is “reasonable.” (In our computations, $\ell = 1$.) When $\sqrt[\ell]{\text{NFS}}$ is run in the ℓ -target version of DT-RSA, the obvious approach is to run precomputation once (taking time T_{precomp}), and the individual forgery ℓ times (taking time $\ell \cdot T_{\text{indiv}}$). Plugging in the relevant costs analyzed by JNT, we have:

$$\begin{aligned} r &= L_N \left[1/3, (8/9)^{1/3} + o(1) \right] \\ T_{\text{precomp}} &= L_N [1/3, 1.577 + o(1)] \\ T_{\text{indiv}} &= L_N \left[1/3, (32/9)^{1/3} + o(1) \right] \end{aligned}$$

For the minimum nontrivial value of ℓ ($\ell = r + 1$) in the OM-RSA game, we have that adversary A attacking OM-RSA takes time:

$$T_{\text{precomp}} + \ell \cdot T_{\text{indiv}} \approx L_N [1/3, 2.488 + o(1)]$$

because $L_N(\alpha, c_1) \cdot L_N(\alpha, c_2) = L_N(\alpha, c_1 + c_2)$. Recall that the GNFS takes time $L_N(1/3, c + o(1))$ with $c = (64/9)^{1/3} \approx 1.923$.

Although it could be argued that $\sqrt[\ell]{\text{NFS}}$ thus yields a subexponential-time attack on OM-RSA, that attack is much worse than factoring N in all cases where OM-RSA is nontrivial. The mismatch comes from the query accounting: $\sqrt[\ell]{\text{NFS}}$ requires many queries to able to forge *any single* target, while OM-RSA expects ℓ forgeries from $\ell - 1$ queries. One might hope that $\sqrt[\ell]{\text{NFS}}$ could be run with fewer queries, or be modified to take advantage of the OM-RSA adaptive oracle, but these changes would require different algorithmic ideas and parameter analyses (including memory, if parallelism is used on the ℓ individual forgeries).

5.2 Another type of oracle from JNT'07

As a second revisiting of past work, $\sqrt[\ell]{\text{NFS}}$ does *not* achieve SNFS complexity when the oracle is restricted to AMR (affine modular root) queries. This difficulty is pointed out in [42, §4.3], and is due to the fact that an AMR oracle imposes a strong constraint on the choice of the algebraic setting of GNFS. The best complexity in that case is given as $L_N(1/3, 6^{1/3} + o(1))$ by [42, §4.3], and the (nontrivial) details to achieve that are only given in the ePrint version [43]. Since $6^{1/3} \approx 1.817$, we expect that variant to be much less practical than the one discussed in this work, and perhaps even barely competitive with factoring.

The AMR setting best matches the PKCS#1v1.5 signature padding that was until recently ubiquitous in practice for RSA signatures. Unfortunately this asymptotic running time precluded implementing the attack with a more realistic padding oracle given our limited computational resources.

6 Scaling the attack to larger moduli

We can attempt to apply the asymptotic complexity formulas to extrapolate bit securities for larger key sizes than we were able to run, but there are a few difficulties. First, ignoring the hidden constants and $o(1)$ terms in the L notation is a bodacious simplification [46] that can hide real differences in practice; second, we have no guarantees that we have reached asymptotic behavior at 1024 bits; and third, we have no guarantee that our parameter choices for the computations we ran were at all optimal. In fact, during the course of this work, we encountered numerous bugs and infelicities across the software we used—including **CADO-NFS**, **SageMath**, and our own code. It is likely that additional issues remain undetected, which could potentially lead to an overestimation of the costs.

With those caveats in place, we extrapolated bit securities of this algorithm for larger key sizes using two methods. In the first method, we computed the multiplicative constant scale factor to recover the generally accepted bit strength for RSA key sizes using the asymptotic running time of the general number field sieve for factoring (blissfully ignoring the $o(1)$ terms), and then applied this constant to the $L_N(1/3, 1.577)$ overall running time of our algorithm. Second, we estimated the multiplicative factor for the cycle count of our 1024-bit computation on a 2.3GHz CPU processor and scaled it up to larger bit sizes using the asymptotic complexity formula. Surprisingly, these values were in fairly close alignment, although extrapolation from the smaller computations we did was less effective in predicting our 1024-bit running times.

To estimate the number of queries required, we applied the asymptotic formula for queries to the empirical value for 1024, ignoring $o(1)$ terms and con-

Key length	Bit security (GNFS)	Bit security ($\sqrt[5]{\text{NFS}}$)	$\log_2(\text{queries})$
1024	80	65	32
2048	112	90	43
3072	128	105	51
4096	144–152	119	57

Fig. 7. Extrapolated security estimates for $\sqrt[5]{\text{NFS}}$. We compare the generally accepted bit security estimates for RSA key sizes based on the complexity of GNFS factoring to the extrapolated running time of the $\sqrt[5]{\text{NFS}}$ algorithm. For the 1024-bit computation that we were able to run, the total cycle count of our computation was quite close to the 2^{65} algorithmic complexity estimate.

stants. As we discussed in Section 4.3, there is a fair amount of leeway in algorithmic parameter selection, and an attacker could make various tradeoffs to somewhat reduce query complexity, online computation cost, or precomputation cost as desired. We give estimated bit complexities in Figure 7.

7 Analysis of concrete attack scenarios

It is natural to ask whether the raw RSA oracle required by our attack actually arises in realistic use scenarios, and if so, what the impact of the security estimates we give is on these applications. As shown in Figure 7, our attack reduces RSA bit security estimates by 15–30 bits for RSA key sizes in use.

The vast majority of RSA use that we are aware of in the modern era is for digital signatures using PKCS#1v1.5 or RSA-PSS padding. As noted in Section 5.2, an RSA oracle that applies such padding does not seem to provide much of an asymptotic speed improvement over factoring for signature forgery. However, raw RSA oracle queries do arise in practical uses of RSA.

7.1 HSMs and hardware-backed keys

HSMs are a popular solution for enterprise or high-value cryptographic keys. Certificate authorities, for example, typically store their key material in HSMs, which are then used to generate signatures on customer/subordinate certificates. Let’s Encrypt [37] describes their use in detail. Code signing is another typical application. For instance, Mitsubishi Electric Sanda Works uses HSMs in the process of writing software to electronic control units, a central component of cars [63]. HSMs can add a layer of hardware protection in situations where a server may be untrusted; HSMs are used as part of WhatsApp’s end-to-end encrypted backup system [24] to run the server-side code for an OPRF where users can recover their messages with a short PIN.

In all of the above use cases, application-specific server software interacts with the HSM through the HSM’s API. The use of the HSM to define a trust boundary around the cryptographic secrets and operations implicitly defines a threat model where the server may be partially compromised or untrusted.

Why would raw RSA be enabled? A reasonable question to ask is why HSMs would support the raw RSA oracle queries that we require for our attack, and why users would enable this mode (and, for example, disable FIPS mode).

One possible reason would be to use the HSM with an RSA padding mode that is not supported by the HSM API. The PKCS #11 specification supports PKCS#1v1.5 padding and RSA-PSS for signatures. However, there are a number of other legacy and modern RSA padding modes, including:

- *ISO 9796-2* [38] padding, used for legacy active authorization (AA) in e-passports and inspection systems according to ICAO [33] and in digital tachographs (devices that track professional vehicles) [19]

- *ANSI X9.31 RSA*, standardized in IEEE 1363-2000 (under the name RSA1 with EMSA2), ISO/IEC 14888-3, and FIPS 186-4 [34, 39, 52, 44]
- *RSA-FDH* (Full Domain Hash), used in GNU Taler and RFC 9381 [31] for an RSA-based verifiable random function

While some of the above standards have been superseded, phasing out hardware that implements them is an ongoing process. Smaller-scale systems like Android KeyMint [2] allow API calls to unpadded RSA decryption. The HSM APIs offer the ability to hash the message on or off the device; for long messages it is likely more efficient to conserve bandwidth by hashing the message off the device and submitting only the hash to the HSM to be signed under the chosen padding scheme, which may expose signature oracles with alternative padding. Under this usage pattern, other padding modes such as `CKM_RSA_PKCS` would expose potentially interesting padding oracles.

7.2 Blind signatures

The $\sqrt[N]{\text{NFS}}$ algorithm applies in any scenario where a raw RSA oracle is available, and the public key (N, e) is known sufficiently far in advance to perform the precomputation before making the oracle queries. RSA blind signatures provide exactly such oracle access to the attacker, allowing it to make (blinded) oracle queries that are indistinguishable from the valid blinded signature queries.

The IRTF CFRG has formalized RSA blind signatures as “RSABSSA” in RFC 9474 [25]. Lysanskaya [49] proved that RSABSSA is secure in the ROM under the OM-RSA assumption, but our attack shows that the OM-RSA assumption may not accurately capture threat models. The section on “security considerations” in RFC 9474 raises the risk of arbitrary RSA signing oracles, and suggests that zero-knowledge proofs could prove well formed-ness of messages, but leaves this out of scope. Blind RSA is often used because it is fast and cheap.

Our estimates suggest that 2048-bit RSA provides too low of a security margin for blind signatures, although the 2^{90} precomputation precludes an immediate threat. Production schemes using blind RSA may consider rotating keys frequently as a short-term mitigation, transitioning to larger key sizes in the medium term, and adding ZK proofs or transitioning to post-quantum schemes in the long term.

Privacy Pass. In the Privacy Pass protocol, clients can pass an attestation check (e.g., solving a CAPTCHA or Apple device attestation) to receive anonymous tokens which can then be used to prove that they are human when visiting a web page [21, 16, 23]. The publicly verifiable token variant, which is implemented by Apple Private Access Tokens and used in production by Fastly, Persona, and Cloudflare, uses blind RSA. An original Cloudflare Privacy Pass implementation issued 30 tokens per CAPTCHA solved [22], which means that for 2048-bit RSA, a single attacker would need to solve $2^{43}/30 \approx 2^{38}$ CAPTCHAs before the key rotation epoch ends, which could be hours [32] to weeks [17]. However,

the number of queries is less daunting when compared with the scale of modern internet infrastructure: Cloudflare has said publicly that they handle more than 7 trillion $\approx 2^{43}$ HTTP requests per day [10], and issue billions of Privacy Pass tokens per day [51]. Apple appears to rate-limit token issuance to one per minute [11], so completing 2^{43} queries would require 17 million years to complete on a single device. Parallelized across the 2.3 billion active Apple devices that Apple claimed in a 2025 earnings call [18] it would take 2.3 days to complete 2^{43} queries at a rate of one per minute per device. Persona charges per API call; at their published rate of \$1.50 per call [58] the cost to carry out the queries would be \$13 trillion.

Apple’s iCloud Private Relay [4] also documents a protocol which uses blind RSA signatures to anonymously authorize web users.

Taler. GNU Taler is an electronic payment system based on Chaumian e-cash. In the protocol, coins are minted via the exchange generating blind signatures using a denomination-specific key. Spenders unblind the signatures in order to spend a coin. In the Taler specification [26], this signing procedure is instantiated using RSA-FDH blind signatures, and timings are provided for 2048 and 1024-bit RSA keys. An adversary carrying out our attack would be able to forge signatures for the denomination-specific key that they attacked, using the blind signature generated by the exchange as the signing oracle in the attack, and afterward be able to spend un-minted coins. For 2048-bit RSA, our estimates imply that this would require, for example, minting \$88B of \$0.01 coins before being able to forge at will.

Taler also now supports clause blind Schnorr signatures [29], which would not be vulnerable to our attack.

7.3 Bleichenbacher RSA padding oracle vulnerabilities

Even if an implementation does not support raw RSA, an attacker may be able to stack other cryptographic or implementation vulnerabilities to obtain a raw RSA oracle. Consider an RSA implementation that only supports PKCS#1v1.5 padding for decryption, but is vulnerable to a Bleichenbacher padding oracle. An attacker could take the direct route and use this oracle to decrypt individual ciphertexts of their choice in an online attack model at a cost of 10^4 – 10^6 oracle queries per decryption (depending on the oracle strength), but would lose this ability if the vulnerability is ever patched. An attacker with an eye towards maintaining future decryption abilities for a key could use the Bleichenbacher padding oracle to compute each oracle query in our attack, at a much increased attack cost, but afterward could more cheaply decrypt ciphertexts.

7.4 1024-bit RSA in the modern era

RSA signatures remain commonly used in practice, and 1024-bit keys can still be found in active use. To our knowledge, all of the protocols below use PKCS#1v1.5 or RSA-PSS padding for signatures, and none of the protocols themselves provide an obvious raw RSA signature oracle to carry out the attack.

TLS. RSA remains a common signature algorithm for TLS both for handshake and certificate signatures. We carried out a ZGrab scan of 421 million domain names extracted from certificate transparency logs from the second half of 2025 and the first half of 2026. Of the 208 million successful TLS handshakes, 55.6% of the host certificates used RSA keys to sign the transcript, of which 99 million (85.7%) were 2048-bit and 17,455 were 1024-bit moduli.

OAuth 2.0. The RS256 algorithm (RSA signatures using PKCS#1v1.5 padding and SHA-256) is a recommended algorithm to implement for the JWT tokens used in OAuth. There is evidence of 1024-bit RSA used for OAuth: documentation from Atlassian updated in September 2026 instructs users to generate 1024-bit RSA keys [5].

DNSSEC. We queried the records for the 1438 TLDs listed in the root zone on September 8, 2026 and received DNSKEY responses for 1361 TLDs. Of these, 1212 (89%) of the ZSKs (Zone-Signing Keys) were 1024-bit RSA.

DKIM. We ran DNS queries on 1048 domains scraped from `lore.kernel.org` emails. All but 19 used RSA keys; 33% of the RSA keys were 1024 bits.

8 Conclusion

In conclusion, our attack illustrates that RSA may provide significantly fewer bits of security in practical attack scenarios than factoring-based estimates would suggest. Our results do not carry over to other popular signature algorithms such as ECDSA or Ed25519. Our projections should be taken as an upper bound on the running time of the attack, since our implementation and parameters are likely to be far from optimal. The current post-quantum transition is a perfect opportunity to adopt cryptography with modern security guarantees.

Acknowledgments

We are grateful to Henry Corrigan-Gibbs for asking the question that launched this entire research project, to Matt Green for extensive help and donating one of the Luna HSMs that we used for the computation, to Jade Philipoom, Fisher Jepsen, Dominic Rizzo, and the team at ZeroRISC for giving us access to one of their Luna HSMs that we used for the computation, to Cindy Moore for logistical and sysadmin help, to Sofia Celi, Henrique Kawakami, Jim Goodman, Keegan Ryan, and anonymous parties for helpful suggestions and equipment donations. L.S. was funded by a Qualcomm fellowship. M.H. was supported by a Google Fellowship and a gift from Zama. A.S. and N.H. were supported by NSF grant no. DMS-2411704. N.H. completed the writing of this paper while a visiting researcher at Inria Saclay.

This work was supported by the PEPR project ANR-PECY-22-0010. High performance computing resources were partially provided by Inria. Some of these

resources were supported by the French “Ministère de l’Enseignement Supérieur et de la Recherche,” by the “Conseil Régional Grand-Est,” and by the European Union through the “Cyber-Entreprises” project.

Experiments presented in this paper were carried out using the Grid’5000 testbed, supported by a scientific interest group hosted by Inria and including CNRS, RENATER, and several universities as well as other organizations (see <https://www.grid5000.fr>), as well as machines donated by Cisco many years ago that currently live at UCSD, and machines made available to us by Jonathan Smith at Penn. We are especially grateful to `lattice22` (for everything) and `lattice17` (for the winning root).

This work was done without the use of generative AI. We did all coding and writing entirely by hand.

References

1. Aggarwal, D., Maurer, U.: Breaking RSA generically is equivalent to factoring. In: Joux, A. (ed.) EUROCRYPT 2009. LNCS, vol. 5479, pp. 36–53. Springer, Berlin, Heidelberg (Apr 2009). https://doi.org/10.1007/978-3-642-01001-9_2 (Cited on 3.)
2. Android Open Source Project: Features, <https://source.android.com/docs/security/features/keystore/features>, visited on September 14, 2026 (Cited on 30.)
3. Aoki, K., Franke, J., Kleinjung, T., Lenstra, A.K., Osvik, D.A.: A kilobit special number field sieve factorization. In: Kurosawa, K. (ed.) ASIACRYPT 2007. LNCS, vol. 4833, pp. 1–12. Springer, Berlin, Heidelberg (Dec 2007). https://doi.org/10.1007/978-3-540-76900-2_1 (Cited on 6, 7.)
4. Apple: iCloud Private Relay Overview, https://www.apple.com/privacy/docs/iCloud_Private_Relay_Overview_Dec2021.PDF, visited on September 15, 2026 (Cited on 31.)
5. Atlassian: OAuth, <https://developer.atlassian.com/server/jira/platform/oauth/> (Cited on 32.)
6. Barker, E., Roginsky, A.: Transitions: Recommendation for transitioning the use of cryptographic algorithms and key lengths. Tech. Rep. NIST Special Publication 800-131A, National Institute of Standards and Technology (NIST), Gaithersburg, MD (January 2011), <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-131a.pdf> (Cited on 2.)
7. Barker, E., Roginsky, A.: Transitions: Recommendation for transitioning the use of cryptographic algorithms and key lengths (revision 3, initial public draft). Tech. Rep. NIST Special Publication 800-131A Rev. 3, National Institute of Standards and Technology (NIST), Gaithersburg, MD (October 2024), <https://csrc.nist.gov/pubs/sp/800/131/a/r3/ipd> (Cited on 2.)
8. Bellare, M., Namprempre, C., Pointcheval, D., Semanko, M.: The one-more-RSA-inversion problems and the security of Chaum’s blind signature scheme. *Journal of Cryptology* **16**(3), 185–215 (Jun 2003). <https://doi.org/10.1007/s00145-002-0020-1> (Cited on 3, 7, 25, 26.)
9. Bellare, M., Riepel, D., Shea, L.: Intermundium-DL: Assessing the resilience of current schemes to discrete-log-computation attacks on public parameters. In: Jager, T., Pan, J. (eds.) PKC 2025, Part IV. LNCS, vol. 15677, pp. 145–178. Springer,

- Cham (May 2025). https://doi.org/10.1007/978-3-031-91829-2_5 (Cited on 7.)
10. Belson, D.: The 2025 Cloudflare Radar Year in Review: The rise of AI, post-quantum, and record-breaking DDoS attacks (12 2025), <https://blog.cloudflare.com/radar-2025-year-in-review/> (Cited on 3, 31.)
 11. Bendell, C.: Private Access Tokens, <https://github.com/colinbendell/private-access-token#private-access-tokens> (Cited on 31.)
 12. Bernard, O., Fouque, P., Lesavourey, A.: Computing e -th roots in number fields. In: Chowdhury, R., Pissis, S.P. (eds.) Proceedings of the 26th Symposium on Algorithm Engineering and Experiments, ALENEX 2024, Alexandria, VA, USA, January 7-8, 2024. pp. 207–219. SIAM (2024). <https://doi.org/10.1137/1.9781611977929.16>, <https://hal.science/hal-04832783v1/document> (Cited on 13.)
 13. Bertocci, V.: JSON Web Token (JWT) Profile for OAuth 2.0 Access Tokens. RFC 9068 (Oct 2021). <https://doi.org/10.17487/RFC9068>, <https://www.rfc-editor.org/info/rfc9068/> (Cited on 2.)
 14. Boudot, F., Gaudry, P., Guillevis, A., Heninger, N., Thomé, E., Zimmermann, P.: Comparing the difficulty of factorization and discrete logarithm: A 240-digit experiment. In: Micciancio, D., Ristenpart, T. (eds.) CRYPTO 2020, Part II. LNCS, vol. 12171, pp. 62–91. Springer, Cham (Aug 2020). https://doi.org/10.1007/978-3-030-56880-1_3 (Cited on 6, 11, 16, 18.)
 15. Boudot, F., Gaudry, P., Guillevis, A., Heninger, N., Thomé, E., Zimmermann, P.: Nouveaux records de factorisation et de calcul de logarithme discret. Techniques de l'Ingénieur p. 17 (2021) (Cited on 2.)
 16. Celi, S., Davidson, A., Valdez, S., Wood, C.A.: Privacy Pass Issuance Protocols. RFC 9578 (Jun 2024). <https://doi.org/10.17487/RFC9578>, <https://www.rfc-editor.org/info/rfc9578/> (Cited on 30.)
 17. Chunhanya, P., Faz-Hernández, A., Celi, S.: Privacy Pass v3: the new privacy bits (10 2021), <https://blog.cloudflare.com/privacy-pass-v3/> (Cited on 30.)
 18. Clover, J.: Apple Now Has More Than 2.35 Billion Active Devices Worldwide (1 2025), <https://www.macrumors.com/2025/01/30/apple-active-devices-worldwide-record/> (Cited on 31.)
 19. Commission Implementing Regulation (EU) 2016/799: Regulations. Official Journal of the European Communities (02 2016), <https://eur-lex.europa.eu/legal-content/EN/TXT/PDF/?uri=CELEX:32016R0799> (Cited on 29.)
 20. Cryptographic Token Interface Standard: X.509 (raw) RSA, https://www.cryptsoft.com/pkcs11doc/v220/group__SEC__12__1__12__X__509___RSA.html#CKM_RSA_X_509, visited on September 10, 2026 (Cited on 20.)
 21. Davidson, A., Iyengar, J., Wood, C.A.: The Privacy Pass Architecture. RFC 9576 (Jun 2024). <https://doi.org/10.17487/RFC9576>, <https://www.rfc-editor.org/info/rfc9576/> (Cited on 30.)
 22. Davidson, A.: Privacy Pass – “The Math” (Nov 2017), <https://blog.cloudflare.com/privacy-pass-the-math/>, visited on September 14, 2026 (Cited on 30.)
 23. Davidson, A., Goldberg, I., Sullivan, N., Tankersley, G., Valsorda, F.: Privacy pass: Bypassing internet challenges anonymously. PoPETs **2018**(3), 164–180 (Jul 2018). <https://doi.org/10.1515/popets-2018-0026> (Cited on 30.)
 24. Davies, G.T., Faller, S.H., Gellert, K., Handirk, T., Hesse, J., Horváth, M., Jager, T.: Security analysis of the WhatsApp end-to-end encrypted backup protocol. In: Handschuh, H., Lysyanskaya, A. (eds.) CRYPTO 2023, Part IV. LNCS, vol. 14084, pp. 330–361. Springer, Cham (Aug 2023). https://doi.org/10.1007/978-3-031-38551-3_11 (Cited on 29.)

25. Denis, F., Jacobs, F., Wood, C.A.: RSA Blind Signatures. RFC 9474 (Oct 2023). <https://doi.org/10.17487/RFC9474>, <https://www.rfc-editor.org/info/rfc9474/> (Cited on 30.)
26. Dold, F.: The GNU Taler System: Practical and Provably Secure Electronic Payments. Phd thesis, Université de Rennes 1 (Feb 2019), <https://hal.science/tel-02138082/> (Cited on 31.)
27. Freeman, D.: Pairing-based identification schemes. Cryptology ePrint Archive, Report 2005/336 (2005), <https://eprint.iacr.org/2005/336> (Cited on 7.)
28. Fried, J., Gaudry, P., Heninger, N., Thomé, E.: A kilobit hidden SNFS discrete logarithm computation. In: Coron, J.S., Nielsen, J.B. (eds.) EUROCRYPT 2017, Part I. LNCS, vol. 10210, pp. 202–231. Springer, Cham (Apr / May 2017). https://doi.org/10.1007/978-3-319-56620-7_8 (Cited on 6, 7, 10, 11, 14, 21, 22.)
29. Fuchsbaauer, G., Plouviez, A., Seurin, Y.: Blind Schnorr signatures and signed ElGamal encryption in the algebraic group model. In: Canteaut, A., Ishai, Y. (eds.) EUROCRYPT 2020, Part II. LNCS, vol. 12106, pp. 63–95. Springer, Cham (May 2020). https://doi.org/10.1007/978-3-030-45724-2_3 (Cited on 31.)
30. Gemalto NV: SafeNet Network HSM Administration Guide, https://public.dhe.ibm.com/cloud/bluemix/network/vpx/administration_guide.pdf, visited on September 10, 2026 (Cited on 20.)
31. Goldberg, S., Reyzin, L., Papadopoulos, D., Včelák, J.: Verifiable Random Functions (VRFs). RFC 9381 (Aug 2023). <https://doi.org/10.17487/RFC9381>, <https://www.rfc-editor.org/info/rfc9381/> (Cited on 30.)
32. Hendrickson, S., Wood, C.A.: Privacy Pass Token Expiration Extension. Internet-Draft draft-ietf-privacypass-expiration-extension-00, Internet Engineering Task Force (May 2026), <https://datatracker.ietf.org/doc/draft-ietf-privacypass-expiration-extension-00/>, work in Progress (Cited on 30.)
33. ICAO: Machine readable travel documents; part 11: Security mechanisms for mrtDs. Tech. Rep. Doc 9303 Eighth Edition, 2021, International Civil Aviation Organization (2021), https://www.icao.int/sites/default/files/publications/DocSeries/9303_p11_cons_en.pdf (Cited on 29.)
34. IEEE: Ieee standard specifications for public-key cryptography. IEEE Std 1363-2000 pp. 1–228 (2000). <https://doi.org/10.1109/IEEESTD.2000.92292> (Cited on 30.)
35. Insight.com: Thales Esecurity – Luna Network HSM S750, [https://www.insight.com/en_US/shop/product/908-000364-00-THEI/\(dnu\)thalesecurityinc/908-000364-005-000/Thales-Esecurity-Luna-Network-HSM-S750-Enterprise-16Mb-5-Partitions-SFM-Ready/](https://www.insight.com/en_US/shop/product/908-000364-00-THEI/(dnu)thalesecurityinc/908-000364-005-000/Thales-Esecurity-Luna-Network-HSM-S750-Enterprise-16Mb-5-Partitions-SFM-Ready/), visited on September 10, 2026 (Cited on 19.)
36. Intel: Intel oneAPI Math Kernel Library, <https://www.intel.com/content/www/us/en/developer/tools/oneapi/onemkl.html> (Cited on 12.)
37. Internet Security Research Group: Combined Certificate Policy and Certification Practice Statement Version 5.9 (Aug 2025), <https://letsencrypt.org/documents/isrg-cp-cps-v5.9/#621-cryptographic-module-standards-and-controls>, visited on September 13, 2026 (Cited on 29.)
38. ISO/IEC: Information technology — security techniques — digital signature schemes giving message recovery; part 2: Integer factorization based mechanisms. Tech. Rep. ISO/IEC 9796-2:2010, International Organization for Standardization and International Electrotechnical Commission (2010) (Cited on 29.)
39. ISO/IEC: It security techniques — digital signatures with appendix; part 3: Discrete logarithm based mechanisms. Tech. Rep. ISO/IEC 14888-3:2018, Interna-

- tional Organization for Standardization (ISO) and International Electrotechnical Commission (IEC) (2018) (Cited on 30.)
40. Itprice.com: DELL AB417971, <https://itprice.com/dell/ab417971.html>, visited on September 10, 2026 (Cited on 19.)
 41. Joux, A., Lercier, R., Naccache, D., Thomé, E.: Oracle-assisted static Diffie-Hellman is easier than discrete logarithms. In: Parker, M.G. (ed.) 12th IMA International Conference on Cryptography and Coding. LNCS, vol. 5921, pp. 351–367. Springer, Berlin, Heidelberg (Dec 2009). https://doi.org/10.1007/978-3-642-10868-6_21 (Cited on 7.)
 42. Joux, A., Naccache, D., Thomé, E.: When e -th roots become easier than factoring. In: Kurosawa, K. (ed.) ASIACRYPT 2007. LNCS, vol. 4833, pp. 13–28. Springer, Berlin, Heidelberg (Dec 2007). https://doi.org/10.1007/978-3-540-76900-2_2 (Cited on 2, 6, 7, 8, 25, 27, 38.)
 43. Joux, A., Naccache, D., Thomé, E.: When e -th roots become easier than factoring. Cryptology ePrint Archive, Report 2007/424 (2007), <https://eprint.iacr.org/2007/424> (Cited on 27.)
 44. Kaliski, B.: RSA Digital Signature Standards (Oct 2000), <https://csrc.nist.gov/files/pubs/conference/2000/10/19/proceedings-of-the-23rd-nissc-2000/final/docs/papers/905slide.pdf>, visited on September 14, 2026 (Cited on 30.)
 45. Koblitz, N., Menezes, A.: Another look at non-standard discrete log and Diffie-Hellman problems. *Journal of Mathematical Cryptology* **2**(4), 311–326 (2008). <https://doi.org/doi:10.1515/JMC.2008.014>, <https://doi.org/10.1515/JMC.2008.014> (Cited on 4, 7.)
 46. Le Gluher, A., Spaenlehauer, P.J., Thomé, E.: Refined analysis of the asymptotic complexity of the Number Field Sieve. *Transactions on Mathematical Cryptology* **1**(1), 71–88 (2021) (Cited on 28.)
 47. Lipmaa, H.: On the CCA1-Security of elgamal and damgård’s elgamal. *Cryptology ePrint Archive*, Report 2008/234 (2008), <https://eprint.iacr.org/2008/234> (Cited on 7.)
 48. Lu, E.: Factoring RSA-260, <https://cognition.com/blog/factoring-rsa-260> (Cited on 4, 6.)
 49. Lysyanskaya, A.: Security analysis of RSA-BSSA. In: Boldyreva, A., Kolesnikov, V. (eds.) PKC 2023, Part I. LNCS, vol. 13940, pp. 251–280. Springer, Cham (May 2023). https://doi.org/10.1007/978-3-031-31368-4_10 (Cited on 30.)
 50. Maine, A.: Luna HSM for Government End-of-Sale and End-of-Life Announcement (Jun 2021), <https://www.thalestct.com/luna-hsm-for-government-end-of-sale-and-end-of-life-announcement/>, visited on September 10, 2026 (Cited on 19.)
 51. Meunier, T.: Moving past bots vs. humans (4 2026), <https://blog.cloudflare.com/past-bots-and-humans/> (Cited on 31.)
 52. National Institute of Standards and Technology: Digital signature standard (DSS). Tech. Rep. FIPS PUB 186-4, U.S. Department of Commerce, Washington, D.C. (2013). <https://doi.org/10.6028/NIST.FIPS.186-4> (Cited on 30.)
 53. National Institute of Standards and Technology: Recommendation for pair-wise key-establishment using integer factorization cryptography. Tech. Rep. NIST Special Publication 800-56B, Revision 2, March 2019 (Publication is current as of January 6, 2026), U.S. Department of Commerce, Washington, D.C. (2019). <https://doi.org/10.6028/NIST.SP.800-56Br2> (Cited on 2.)

54. National Institute of Standards and Technology: Transition management plan for post-quantum cryptography. NIST Interagency or Internal Report NIST IR 8547 (Initial Public Draft), U.S. Department of Commerce (2024). <https://doi.org/10.6028/NIST.IR.8547.ipd>, <https://nist.gov> (Cited on 2.)
55. OASIS Open: PKCS #11 Cryptographic Token Interface Current Mechanisms Specification Version 2.40 (April 2015), <https://docs.oasis-open.org/pkcs11/pkcs11-curr/v2.40/os/pkcs11-curr-v2.40-os.html>, oASIS Standard (Cited on 3.)
56. OpenMPI: OpenMPI Library, <https://www.open-mpi.org/> (Cited on 23.)
57. PCI Security Standards Council: Payment Card Industry Data Security Standard (PCI DSS) Version 4.0.1. Tech. rep., PCI Security Standards Council (June 2024), https://www.pcisecuritystandards.org/document_library/ (Cited on 3.)
58. Persona: Plans Overview, <https://help.withpersona.com/articles/6oZbZp7jb7AWGC1F5vpY3K/> (Cited on 31.)
59. SafeNet USB HSM 6.3 Product Documentation: About PED Keys, https://thalesdocs.com/gphsm/luna/6.3/docs/usb/Content/administration/ped_auth/about_ped_keys.htm, visited on September 10, 2026 (Cited on 19.)
60. SafeNet USB HSM 6.3 Product Documentation: How to Use a SafeNet PED, https://thalesdocs.com/gphsm/luna/6.3/docs/usb/Content/administration/ped_auth/how_to_use_a_ped.htm, visited on September 10, 2026 (Cited on 19.)
61. SciPy: SciPy Library, <https://scipy.org/> (Cited on 12.)
62. Thales Group: Pycryptoki, <https://github.com/ThalesGroup/pycryptoki/tree/master>, visited on September 10, 2026 (Cited on 19.)
63. Thales Group: Mitsubishi Electric Sanda Works: Encryption management using Thales Luna HSMs that yield significant improvements in security during the software writing process (Mar 2024), https://cpl.thalesgroup.com/sites/default/files/content/case_studies/field_document/2024-05/luna-hsm-encryption-mitsubishi-electric-sanda-works-cs.pdf, visited on September 13, 2026 (Cited on 29.)
64. The CADO-NFS Development Team: CADO-NFS, an implementation of the Number Field Sieve algorithm (2026), <http://cado-nfs.inria.fr/> (Cited on 4, 8.)
65. The Flatiron Institute: sparse_dot Library, https://github.com/flatironinstitute/sparse_dot (Cited on 12.)
66. The PARI Group, Univ. Bordeaux: PARI/GP version 2.17.1 (2026), available from <http://pari.math.u-bordeaux.fr/> (Cited on 8.)
67. The Sage Developers: SageMath, the Sage Mathematics Software System (Version 10.7) (2026), <https://www.sagemath.org> (Cited on 8.)
68. Thomé, E.: Square root algorithms for the number field sieve. In: Özbudak, F., Rodríguez-Henríquez, F. (eds.) *Arithmetic of Finite Fields*. pp. 208–224. Springer Berlin Heidelberg, Berlin, Heidelberg (2012). https://doi.org/https://doi.org/10.1007/978-3-642-31662-3_15 (Cited on 12, 13.)
69. Thomé, E.: The Number Field Sieve (lecture notes) (2022), <https://cseweb.ucsd.edu/classes/wi22/cse291-14/> (Cited on 6, 8.)
70. Weis, S.: About the new RSA-260 record, <https://x.com/sweis/status/2095570645505700165> (Cited on 4.)
71. Weis, S.: RSA-896, <https://saweis.net/posts/rsa-896.html> (Cited on 6.)
72. Wikipedia: Integer factorization records, https://en.wikipedia.org/wiki/Integer_factorization_records (Cited on 6.)

A The $\sqrt[e]{NFS}$ algorithm, for poets

With billions of queries, and RSA signatures,
 we throw number theories at very large integers.
 From 2007 [42] we use NFS sorcery—
 to conjure, in months, a signature forgery!
 Worry not, reader, we don't factor N ;
 we compute eth roots with a different descent.

Enter the target: RSA ten-twenty-four,
 the key safe inside an HSM's core.
 But—notice now—this exposes an oracle,
 and a special attack that's more than rhetorical.
 SNFS is the complexity we glean,
 a speedup quite doable on academic machines.

By this time, reader, it is high time to give
 a poetic account of our number field sieve.
 First come I , A and B , an N and an e ,
`lim` and `powlim`, and three LPBs;
 a degree around six, a skewed f and a g .
 With the diamond thus drawn, the parameters teed,
 let us loose `las`, and `bwc`!

Our journey begins with precomputation,
 where we one-sided sieve on many workstations.
 We relate our relations to things in $\mathbb{Z}_N$,
 make all our queries, and queue up a huge M .

At this point we pick any target to break;
 an equation mod e is the next step to make.
 We descend into smoothness and madness to boot,
 ere Dr. Montgomery computes an eth root.
 But it's not quite the end, δ 's not quite a unit,
 we need one more job to finally do it:
 p -adic ascent in a critical race,
`hybrid.eth.root` lifts us out of this case!
 With this root in the field, and our earlier yield,
 we raise t to the d , though p remains sealed!